LAPORAN TUGAS BESAR II

IF3070 DASAR INTELIGENSI ARTIFISIAL

IMPLEMENTASI ALGORITMA PEMBELAJARAN MESIN

Dosen: Dr. Nur Ulfa Maulidevi, S.T, M.Sc.



Disusun Oleh :

Kelompok 31 Welfarella

Favian Izza Diasputra	18222070
Henrycus Hugatama Risaldy	18223008
Farella Kamala Budianto	18223046
Kenlyn Tesalonika Winata	18223098

SISTEM DAN TEKNOLOGI INFORMASI

INSTITUT TEKNOLOGI BANDUNG

DESEMBER 2025

DAFTAR ISI

BAB I **3**
LATAR BELAKANG **3**
BAB II **4**
DASAR TEORI **4**
2.1 Decision Tree Learning 4
2.2 Logistic Regression 7
2.3 K-Nearest Neighbors 10
BAB III **15**
DATA INFORMATION **15**
Tabel 1 Deskripsi kolom pada dataset 15
BAB IV **17**
EXPLORATORY DATA ANALYSIS **17**
4.1 Data size 17
4.1.1 The size of the data 17
4.1.2 Categorization of Features 18
4.1.3 Data type and number of unique values 19
Tabel 2 Data Type, Number, and Sample of Unique Values 19
Tabel 3 Data Type, Number of Unique Values 20
4.1.4 Values for numerical features 21
Tabel 4 Values for Numerical Features 22
4.1.5 Correlation between categorical and label 23
4.1.6 Correlation between numerical and label 25
4.2 Missing values for each feature 26
4.2.1 Missing values for each feature 26
4.2.2 Outliers for each feature 27
4.2.5 Correlation between features and the target variable 32
4.3 Split Training Set, Validation Set, Testing Set 34
BAB V **35**
CLEANING DAN PREPROCESSING DATA **35**
5.1 Data Cleaning 35
5.1.1 Handling Missing Data 35
5.1.2 Dealing with Outliers 37
5.1.3 Remove Duplicates 38
5.1.4 Feature Engineering 38
5.2 Data Preprocessing 40
5.2.1 Feature Scaling 40
5.2.2 Feature Encoding 40
5.2.3 Handling Imbalanced Dataset 42
5.2.4 Data Normalization 42
5.2.5 Dimensionality Reduction 43

BAB VI **44**
IMPLEMENTASI ALGORITMA **44**
6.1 Algoritma Decision Tree Learning 44
6.2 Algoritma Logistic Regression 51
BAB VII **63**
DATA SPLITTING, PREPROCESSING PIPELINE, SUBMISSION, dan BONUS **63**
7.1 Data Splitting 63
7.2 Preprocessing Pipeline 63
7.3 Submission 65
7.4 Bonus DTL 66
BAB VIII **71**
PERBANDINGAN HASIL PREDIKSI **71**
7.1 Perbandingan Hasil Prediksi Algoritma Decision Tree Learning 71
7.2 Perbandingan Hasil Prediksi Algoritma Logistic Regression 72
7.3 Perbandingan Hasil Prediksi Algoritma K-Nearest Neighbors 73
BAB IX **75**
PENUTUP **75**
8.1 Kesimpulan 75
8.2 Kontribusi Setiap Anggota Kelompok 76
Tabel 5 Kontribusi Setiap Anggota Kelompok 76
REFERENSI **79**

BAB I
LATAR BELAKANG

Perkembangan teknologi digital membuat transaksi *online* semakin menjadi bagian penting dalam kehidupan sehari-hari. Berbagai aktivitas seperti belanja, pembayaran layanan, dan transfer dana kini dilakukan secara instan melalui platform digital. Namun, seiring meningkatnya volume transaksi, risiko keamanan juga ikut meningkat. Ancaman seperti pencurian identitas, carding, aktivitas bot, dan transaksi palsu semakin sering terjadi dan dapat menyebabkan kerugian bagi pengguna maupun penyedia layanan.

Berdasarkan meningkatnya risiko penipuan dalam transaksi digital, kemampuan untuk mendeteksi apakah sebuah transaksi termasuk *fraud* menjadi sangat penting. Perusahaan sekarang akan mengandalkan model *machine learning* untuk membaca pola perilaku pengguna, menganalisis risiko perangkat, memahami konteks lokasi, dan mempelajari dinamika transaksi. Tantangannya adalah data transaksi di dunia nyata umumnya tidak bersih dan kurang ideal sehingga dataset sering kali berisi *missing values, outliers,* interaksi fitur yang kompleks, serta ketidakseimbangan kelas yang mana jumlah transaksi normal jauh lebih dominan dibandingkan jumlah transaksi *fraud.*

Pada tugas besar ini, mahasiswa IF3070 diberikan dataset yang menyerupai karakteristik data transaksi nyata, dan diminta untuk membangun model *machine learning* yang mampu mengidentifikasi *fraud* secara akurat. Dalam pengerjaan, kami tidak hanya berfokus pada pemilihan algoritma, tetapi juga mencakup pembersihan data, penanganan *outliers,* transformasi fitur, pemilihan metrik evaluasi yang sesuai, serta strategi mengatasi *imbalanced class* supaya model tidak bias terhadap kelas mayoritas.

BAB II
DASAR TEORI

2.1 Decision Tree Learning

Decision Tree Learning (DTL) merupakan salah satu algoritma *supervised learning* yang digunakan untuk klasifikasi atau regresi dengan cara membuat pohon keputusan (*decision tree*). Algoritma ini akan memetakan data ke dalam bentuk pohon, di mana setiap *node* akan merepresentasikan pengujian terhadap suatu atribut, setiap cabangnya akan mempresentasikan satu kemungkinan nilai dari atribut tersebut, setiap *leaf node* akan merepresentasikan hasil klasifikasi atau label kelas yang merupakan nilai dari prediksi akhir, dan setiap jalur dari akar ke daun akan merepresentasikan aturan *if-then* yang dilakukan.

Decision Tree Learning cocok digunakan untuk permasalahan klasifikasi atau regresi dengan data yang direpresentasikan dalam bentuk atribut dan nilai, memiliki target hasil yang diskrit atau numerik, dan memungkinkan adanya data yang mengandung *noise* ataupun nilai yang hilang. DTL juga banyak digunakan karena hasil modelnya yang mudah diinterpretasikan dalam bentuk aturan keputusan.

Dalam *decision tree*, data direpresentasikan menjadi pasangan atribut-nilai (*attribute-value pairs*) yang akan melakukan pembelajaran secara *top-down* dan *greedy*. Proses akan dimulai dengan melakukan pemilihan atribut terbaik untuk menjadi akar pohon (*root node*), kemudian dilanjutkan dengan memecahkan data ke dalam cabang-cabang berdasarkan nilai atribut tersebut. Proses ini dilakukan secara rekursif hingga seluruh data ditempatkan dengan baik atau tidak ada atribut yang tersisa untuk dipecah lagi.

Decision tree learning memiliki beberapa algoritma dasar yang sering digunakan seperti ID3 (*Iterative Dichotomiser* 3), C4.5, dan CART (*Classification and Regression Tree*). ID3 merupakan algoritma yang memilih atribut terbaik di setiap *node* berdasarkan nilai *information gain* yang dihitung menggunakan konsep *entropy* dan umumnya digunakan

untuk klasifikasi atribut kategorikal dan menghasilkan pohon dengan jumlah cabang sesuai dengan banyaknya nilai atribut yang digunakan. Algoritma C4.5 merupakan pengembangan dari ID3, di mana algoritma ini menggunakan *gain ratio* sebagai kriteria dalam pemilihan atribut untuk mengurangi bias terhadap atribut yang memiliki banyak nilai.

Entropy sendiri merupakan ukuran ketidakpastian dari suatu kumpulan data yang digunakan untuk mengukur seberapa tidak murni suatu node terhadap kelas targetnya. Sehingga, semakin tinggi nilai *entropy*, semakin tidak teratur distribusi kelas dalam data tersebut. Secara matematis, *entropy* dirumuskan sebagai berikut:

$$Entropy(S) = -\sum_{i=1}^{c} p_i log_2 p_i$$

dengan:

- S merupakan himpunan data,
- c merupakan jumlah kelas,
- p_i merupakan proporsi data pada kelas ke-i.

Nilai *entropy* akan bernilai 0 jika seluruh data berada pada satu kelas dan akan bernilai maksimum jika distribusi data merata di seluruh kelas.

Terdapat juga nilai *information gain* yang digunakan dalam menentukan atribut terbaik yang akan berperan sebagai pemisah suatu *node* untuk mengukur seberapa besar penurunan nilai *entropy* setelah data dibagi berdasarkan suatu atribut. Secara matematis, *information gain* dirumuskan sebagai berikut:

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

dengan:

- A merupakan atribut yang diuji,
- S_v merupakan subset data dengan nilai atribut $A = v$.

Atribut dengan nilai *information gain* tertinggi akan dipilih sebagai *node* pemisah karena dianggap paling efektif dalam mengurangi ketidakpastian klasifikasi data.

Selain algoritma berbasis entropi tersebut, terdapat juga CART yang merupakan algoritma *Decision Tree Learning* yang menggunakan *gini impurity* sebagai kriteria pemilihan atributnya dan akan selalu membentuk pohon biner, di mana setiap *node* hanya akan memiliki dua cabang yang membuatnya lebih sederhana dan konsisten.

Gini impurity merupakan ukuran ketidakmurnian data pada suatu *node*, nilai yang kecil menunjukkan bahwa distribusi data pada node tersebut lebih homogen atau didominasi oleh satu kelas tertentu. Secara matematis, *gini impurity* dirumuskan sebagai berikut:

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^{\,2}$$

dengan:

- S merupakan himpunan data,
- c merupakan jumlah kelas,
- p_i merupakan proporsi data pada kelas ke-i.

Dalam proses pemilihan atribut, CART tidak hanya melihat nilai *gini impurity*, tetapi juga menghitung nilai *gini* setelah data dibagi ke dalam beberapa subset. Nilai ini disebut sebagai *gini split* atau *weighted gini* yang dirumuskan sebagai berikut:

$$Gini_{split}(S) = \sum_{j=1}^{k} \frac{|S_j|}{|S|} Gini(S_j)$$

dengan:

- S merupakan himpunan data pada node awal,
- S_j merupakan subset ke-j hasil pemisahan,
- k merupakan jumlah subset (pada CART, $k = 2$).

Setelah itu, CART akan memilih atribut dan split yang meminimalkan nilai *gini* karena pemisahan tersebut menghasilkan *node-node* dengan tingkat kemurnian yang lebih tinggi. Pada tugas besar ini, diimplementasikan algoritma C4.5 karena merupakan materi yang

diajarkan di kelas dan dari hasil percobaan, algoritma ini menghasilkan prediksi yang lebih baik dari algoritma yang berbasis pada *gini impurity*.

2.2 Logistic Regression

Logistic Regression merupakan algoritma *supervised learning* yang digunakan secara luas untuk pemodelan klasifikasi, khususnya pada permasalahan klasifikasi biner (*binary classification*) di mana target variabel memiliki dua kemungkinan nilai, seperti *fraud* (1) atau *normal* (0). Meskipun memiliki nama "regression", algoritma ini pada dasarnya adalah pengklasifikasi linear yang mengestimasi probabilitas bahwa sebuah data input termasuk ke dalam kelas tertentu. Algoritma ini bekerja dengan memodelkan hubungan antara fitur-fitur independen dengan logaritma *odds* (*log-odds*) dari variabel target, yang kemudian ditransformasikan menjadi nilai probabilitas menggunakan fungsi non-linear.

Algoritma ini sangat cocok digunakan ketika variabel dependen bersifat kategorikal, namun analisis membutuhkan *output* yang lebih informatif daripada sekadar label kelas, yaitu berupa nilai probabilitas kepercayaan model. Keunggulan utama *Logistic Regression* terletak pada efisiensi komputasinya, kemudahannya untuk diinterpretasikan karena bobot (*weights*) yang dihasilkan berkorelasi langsung dengan tingkat kepentingan fitur, serta kemampuannya untuk diperluas dengan teknik regularisasi guna menangani *overfitting* pada dataset berdimensi tinggi.

Dalam mekanismenya, *Logistic Regression* memulai perhitungan dengan mengkombinasikan input data secara linear dengan parameter bobot (*weights*) dan bias. Kombinasi linear ini sering disebut sebagai *logit* (z). Namun, karena hasil kombinasi linear ini memiliki rentang nilai dari $+\infty$ hingga $-\infty$, nilai tersebut tidak dapat langsung digunakan sebagai probabilitas. Oleh karena itu, diperlukan fungsi aktivasi untuk memetakan nilai tersebut ke dalam rentang [0, 1]. Fungsi yang digunakan dalam algoritma ini adalah fungsi *Sigmoid* atau *Logistic Function*.

Fungsi *Sigmoid* memiliki kurva berbentuk "S" yang halus dan dapat diturunkan (*differentiable*), yang menjadikannya ideal untuk proses optimasi berbasis gradien. Secara

matematis, transformasi dari kombinasi linear menjadi probabilitas dirumuskan sebagai berikut:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

dengan:

- $z = w \cdot X + b$ di mana w adalah vektor bobot, X adalah vektor fitur input, dan b adalah bias,
- e merupakan basis logaritma natural,
- $\sigma(z)$ merupakan estimasi probabilitas bahwa data termasuk dalam kelas positif $(P(y = 1| X))$

Nilai probabilitas yang dihasilkan kemudian dikonversi menjadi keputusan kelas diskrit menggunakan suatu nilai ambang batas (*threshold*). Jika $\sigma(z) \geq threshold$ (umumnya 0.5), data diklasifikasikan sebagai kelas positif, dan sebaliknya.

Untuk melatih model agar menghasilkan prediksi yang akurat, *Logistic Regression* tidak menggunakan metode *Least Squares* seperti pada regresi linear karena fungsi prediksi yang non-linear akan menghasilkan fungsi biaya yang *non-convex* (memiliki banyak minimum lokal). Sebagai gantinya, digunakan fungsi *loss* bernama *Binary Cross-Entropy* atau *Log Loss*. Fungsi ini didasarkan pada prinsip *Maximum Likelihood Estimation* (MLE), yang bertujuan untuk memaksimalkan kemungkinan parameter model menghasilkan data yang diamati. Fungsi ini memberikan penalti yang sangat besar (nilai *loss* tinggi) jika model memprediksi probabilitas yang salah dengan tingkat kepercayaan yang tinggi.

Secara matematis, fungsi *Binary Cross-Entropy* untuk satu sampel data dirumuskan sebagai:

$$L(y, \widehat{y}) = -[y\, log(\widehat{y}) + (1 - y) log(1 - \widehat{y})]$$

dengan:

- y merupakan label kelas sebenarnya (0 atau 1),
- $\widehat{y}$ merupakan probabilitas prediksi dari model ($\sigma(z)$)

Proses pembelajaran model dilakukan dengan meminimalkan rata-rata *loss* pada seluruh dataset pelatihan menggunakan algoritma optimasi *Gradient Descent*. Algoritma ini bekerja secara iteratif memperbarui parameter bobot dan bias dengan bergerak berlawanan arah dari gradien fungsi *loss*. Pada implementasi kasus nyata, khususnya dalam deteksi *fraud* yang memiliki ketidakseimbangan kelas (*imbalanced data*) dan dimensi fitur yang tinggi, *Logistic Regression* sering kali dimodifikasi dengan penambahan teknik *Regularization* dan *Class Weighting*.

L2 Regularization (Ridge) diterapkan untuk mencegah *overfitting* dengan menambahkan suku penalti berupa kuadrat dari besaran bobot ke dalam fungsi biaya. Hal ini memaksa model untuk menjaga nilai bobot tetap kecil, sehingga mengurangi sensitivitas model terhadap *noise* pada data latih. Fungsi biaya total dengan regularisasi L2 dirumuskan sebagai:

$$J(w) = \frac{1}{n}\sum_{i=1}^{n} L(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2n}\sum_{j=1}^{m} w_j^2$$

dengan:

- $J(w)$ merupakan fungsi biaya total (*Total Cost Function*) yang ingin diminimalkan oleh model selama proses pelatihan. Nilai ini merupakan penjumlahan dari rata-rata kesalahan prediksi (*loss*) dan penalti regularisasi.
- n merupakan jumlah total sampel data (*instances*) dalam *dataset* pelatihan. Pembagian dengan n berfungsi untuk menormalisasi nilai *loss* agar skalanya tidak bergantung pada banyaknya data.
- i merupakan indeks yang mengacu pada sampel data spesifik dalam iterasi, berjalan dari 1 hingga n.
- $L(y^{(i)}, \hat{y}^{(i)})$ merupakan fungsi *loss* (seperti *Binary Cross-Entropy*) untuk sampel data ke-i, yang mengukur selisih antara label sebenarnya dengan probabilitas prediksi model.
- $y^{(i)}$merupakan label kelas sebenarnya (*ground truth*) untuk sampel data ke-i. Bernilai 1 untuk kelas positif (*fraud*) dan 0 untuk kelas negatif (*normal*).
- $\hat{y}^{(i)}$ merupakan probabilitas prediksi yang dihasilkan oleh model untuk sampel data ke-i (output dari fungsi sigmoid $\sigma(z)$).
- λ (lambda) merupakan parameter regularisasi (*hyperparameter*) yang mengontrol seberapa kuat penalti diberikan terhadap bobot model. Nilai λyang lebih tinggi akan memaksa bobot menjadi lebih kecil untuk mencegah *overfitting*.

- m merupakan jumlah fitur (*features*) atau dimensi input yang digunakan dalam model.
- j merupakan indeks yang mengacu pada fitur atau bobot spesifik, berjalan dari 1 hingga m.
- w_j merupakan nilai bobot (*weight*) parameter model yang berasosiasi dengan fitur ke-j. Dalam regularisasi L2, nilai ini dikuadratkan (w_j^2) untuk memberikan penalti yang lebih besar pada bobot yang bernilai ekstrem.

Selain itu, teknik *Class Weighting* digunakan untuk mengatasi dominasi kelas mayoritas (*normal*) terhadap kelas minoritas (*fraud*). Dengan teknik ini, fungsi *loss* dimodifikasi agar memberikan bobot kesalahan yang lebih besar pada kelas minoritas. Hal ini mendorong model untuk lebih "memperhatikan" pola pada transaksi *fraud* selama proses pembelajaran, sehingga meningkatkan performa deteksi pada kelas yang lebih jarang muncul tersebut. Pada tugas besar ini, algoritma diimplementasikan dari awal (*from scratch*) dengan mengintegrasikan seluruh komponen tersebut untuk menghasilkan model deteksi *fraud* yang *robust*.

2.3 K-Nearest Neighbors

K-Nearest Neighbors (KNN) merupakan algoritma *supervised machine learning* yang bersifat non-parametrik dan *instance-based* atau *lazy learning*. KNN disebut non-parametrik karena KNN tidak mengasumsikan bentuk distribusi data tertentu, dan disebut *lazy learning* karena algoritma ini tidak membangun model matematis secara eksplisit pada tahap *training*. Pada KNN, seluruh data *training* disimpan dalam kondisi asli, dan tahap pemrosesan baru dilakukan ketika terdapat data baru yang ingin diprediksi. Namun, kekurangan dari pendekatan ini adalah KNN tidak melakukan generalisasi global, melainkan mengambil keputusan berdasarkan kedekatan lokal antara data uji dan data latih.

Pada permasalahan deteksi *fraud,* pendekatan berbasis kemiripan lokal ini umumnya relevan karena banyak kasus *fraud* muncul sebagai pola yang menyerupai transaksi *fraud* sebelumnya, bukan sebagai pola global yang terpisah secara jelas. Secara matematis, misalkan tersedia sebuah dataset *training* seperti ini:

$$D = \{(x1, y1), (x2, y2), ..., (xN, yN)\}$$

dengan:

- D merupakan dataset *train*
- xi merupakan informasi dari transaksi ke-i
- yi merupakan label kelas transaksi ke-i
- N merupakan banyaknya transaksi dalam data *training*
- Label yi = 1 menunjukkan transaksi *fraud,* sedangkan yi = 0 menunjukkan transaksi *non-fraud*

dengan $xi \in \mathbb{R}^n$, yaitu setiap transaksi ke-i direpresentasikan dalam bentuk beberapa nilai numerik dan $yi \in \{0, 1\}$ yaitu label kelas (0 untuk non-*fraud* dan 1 untuk *fraud).* Pada kalimat matematis tersebut, diberikan sebuah data uji berupa xq, dan tujuan KNN adalah menentukan label yq, berdasarkan label transaksi-transaksi dalam dataset *training* yang paling mirip dengan xq. Kemiripan ini diukur melalui perhitungan jarak di ruang fitur.

Dalam KNN, kemiripan antar transaksi diukur menggunakan suatu aturan perhitungan yang disebut dengan metrik jarak. Semakin kecil nilai jarak yang dihasilkan, semakin tinggi tingkat kemiripan antara dua transaksi, sedangkan nilai jarak yang besar menunjukkan bahwa kedua transaksi memiliki perbedaan yang signifikan. Metrik jarak yang paling umum digunakan dalam KNN adalah jarak Euclidean, yang secara matematis didefinisikan sebagai:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (xi - yi)^2}$$

dengan:

- $d(x, y)$ merupakan nilai jarak antara dua transaksi yang sedang dibandingkan
- x merupakan informasi transaksi pertama yang dinyatakan dalam bentuk kumpulan nilai numerik
- y merupakan informasi transaksi kedua yang dinyatakan dalam bentuk kumpulan nilai numerik
- xi merupakan nilai informasi transaksi pertama pada aspek ke-i
- yi merupakan nilai informasi transaksi kedua pada aspek ke-i

- n merupakan banyaknya aspek informasi transaksi yang digunakan

Rumus ini mengukur seberapa besar perbedaan antara dua transaksi dengan membandingkan nilai pada setiap aspek informasi transaksi yang digunakan. Selisih pada setiap aspek tersebut dikuadratkan untuk memastikan seluruh perbedaan bernilai positif sekaligus memberikan penalti yang lebih besar pada perbedaan yang signifikan. Lalu, hasil penjumlahan tersebut diakarkan sehingga skala jarak yang dihasilkan tetap sebanding dengan skala nilai aslinya. Dalam konteks deteksi *fraud,* jarak Euclidean yang kecil menunjukkan bahwa dua transaksi memiliki pola perilaku yang sangat mirip, sedangkan jarak besar menunjukkan adanya perbedaan perilaku transaksi yang signifikan

Selain jarak Euclidean, KNN juga dapat menggunakan jarak Manhattan sebagai alternatif metrik jarak, yang secara matematis didefinisikan sebagai:

$$d(x, y) = \sum_{i=1}^{n} |xi - yi|$$

dengan:

- $d(x, y)$ merupakan nilai jarak antara dua transaksi yang sedang dibandingkan
- x merupakan informasi transaksi pertama yang dinyatakan dalam bentuk kumpulan nilai numerik
- y merupakan informasi transaksi kedua yang dinyatakan dalam bentuk kumpulan nilai numerik
- xi merupakan nilai informasi transaksi pertama pada aspek ke-i
- yi merupakan nilai informasi transaksi kedua pada aspek ke-i
- n merupakan banyaknya aspek informasi transaksi yang digunakan

Berbeda dengan jarak Euclidean, jarak Manhattan menghitung total perbedaan antar transaksi dengan menjumlahkan selisih absolut pada setiap aspek informasi transaksi tanpa melakukan pengkuadratan. Pendekatan ini membuat jarak Manhattan tidak memberikan penalti berlebih pada suatu perbedaan yang sangat besar, sehingga dapat lebih stabil ketika data transaksi mengandung nilai ekstrim. Dalam deteksi *fraud,* jarak

Manhattan dapat digunakan untuk membandingkan transaksi berdasarkan perbedaan langsung pada setiap aspek informasinya.

Selain metrik berbasis selisih nilai, KNN juga dapat menggunakan *cosine similarity* untuk mengukur kemiripan antar transaksi berdasarkan pola informasinya, yang secara matematis didefinisikan sebagai:

$$cos(\theta) = \frac{x \,.\, y}{||x||\,||y||},\ d(x,y) = 1 - cos(\theta)$$

dengan:

- $cos(\theta)$ merupakan nilai *cosine similarity* yang menunjukkan tingkat kemiripan antara dua transaksi
- x merupakan informasi transaksi pertama yang dinyatakan dalam bentuk kumpulan nilai numerik
- y merupakan informasi transaksi kedua yang dinyatakan dalam bentuk kumpulan nilai numerik
- $x \,.\, y$ merupakan hasil perkalian *dot product* antara dua transaksi

Cosine similarity mengukur kemiripan dua transaksi berdasarkan arah representasi informasinya, bukan berdasarkan besar kecilnya nilai antar transaksi. Dua transaksi akan dianggap mirip ketika memiliki pola distribusi informasi yang searah, meskipun nilai-nilai informasinya berada pada skala yang berbeda. Dalam konteks deteksi *fraud,* metrik ini berguna ketika kemiripan pola transaksi lebih penting dibandingkan perbedaan nilai secara langsung, terutama pada data dengan skala yang bervariasi.

Prediksi kelas pada KNN dilakukan melalui mekanisme *majority voting*. Mekanisme ini diartikan bahwa kelas yang paling sering muncul di antara k tetangga terdekat akan dipilih sebagai hasil prediksi. Pada data *fraud* yang tidak seimbang, mekanisme voting mayoritas ini menjadi sumber utama terjadinya *false negative,* karena transaksi *fraud* sering kalah jumlah dibandingkan transaksi normal. Kemudian, KNN juga dapat memberikan skor probabilitas berdasarkan proporsi tetangga *fraud*. Probabilitas sebuah transaksi diklasifikasikan sebagai *fraud* dapat dihitung sebagai:

$$P(fraud \mid Xq) = \frac{jumlah\ tetangga\ fraud}{k}$$

dengan:

- $P(fraud \mid Xq)$ merupakan peluang transaksi yang sedang diperiksa tergolong *fraud*
- Xq merupakan transaksi baru atau data uji yang ingin ditentukan apakah termasuk *fraud* atau tidak
- k merupakan jumlah tetangga terdekat yang digunakan dalam KNN

Nilai probabilitas ini berada pada rentang [0,1], dan memberikan informasi tingkat keyakinan model terhadap prediksi yang dihasilkan. Dalam kasus mendeteksi *fraud,* skor probabilitas ini sering digunakan untuk evaluasi menggunakan metrik ROC AUC serta untuk penyesuaian threshold keputusan.

Meskipun sederhana dan intuitif, KNN memiliki keterbatasan yang cukup signifikan. Salah satu keterbatasannya adalah *curse of dimensionality,* yang mana peningkatan jumlah fitur menyebabkan jarak antar data menjadi semakin sama. Dalam kondisi ini, perbedaan antara tetangga terdekat dan tetangga yang lebih jauh menjadi tidak signifikan, yang mengakibatkan performa KNN menurun, terutama pada dataset berdimensi tinggi. Secara keseluruhan, KNN merupakan algoritma yang sederhana, tetapi cukup efektif dalam hal mendeteksi *fraud*. Dengan *preprocessing* yang tepat, pemilihan parameter yang juga tepat, serta evaluasi yang sesuai, KNN cukup dapat mendeteksi pola *fraud* yang konsisten dan berulang. Namun, keterbatasannya terhadap data berskala besar, berdimensi tinggi, dan pola *fraud* yang benar-benar baru membuat KNN kurang cocok sebagai solusi tunggal.

BAB III
DATA INFORMATION

Dataset yang disediakan merupakan data sintetis berskala besar (100,000 baris dan 31 kolom) yang dibuat sedemikian rupa agar menyerupai pola *fraud* nyata: adanya *outlier*, *missing values,* perilaku mencurigakan, dan interaksi antar-fitur yang kompleks.

Tabel 1 Deskripsi kolom pada dataset

Nama Kolom	Penjelasan
ID	Indeks baris unik (integer) yang dibuat di akhir proses generasi data.
transaction_id	ID unik transaksi (string, format contoh: T0001234).
user_id	ID pengguna yang melakukan transaksi; menghubungkan beberapa transaksi milik user yang sama.
age	Umur pengguna (tahun).
gender	Jenis kelamin pengguna: M (male) atau F (female).
country	Negara tempat transaksi dilakukan (misal: US, UK, ID).
device_type	Jenis perangkat: mobile, desktop, atau tablet.
device_os	Sistem operasi perangkat (Android, iOS, Windows, Linux, MacOS).
transaction_amount	Jumlah uang dalam transaksi.
transaction_type	Jenis transaksi: purchase, transfer, topup, atau withdrawal.
merchant_category	Kategori merchant/toko (electronics, groceries, gas, dll).
time_of_day	Jam transaksi (0–23).
day_of_week	Hari dalam seminggu (0 = Senin, …, 6 = Minggu).
transaction_duration	Durasi proses transaksi (detik), dibangkitkan dengan distribusi eksponensial.
num_prev_transactions	Jumlah transaksi sebelumnya yang pernah dilakukan user.
avg_transaction_amount	Rata-rata jumlah transaksi historis user.

std_transaction_amount	Standar deviasi jumlah transaksi historis user.
transactions_last_24h	Jumlah transaksi user dalam 24 jam terakhir.
transactions_last_1h	Jumlah transaksi user dalam 1 jam terakhir.
failed_login_attempts	Jumlah percobaan login gagal sebelum transaksi ini.
ip_risk_score	Skor risiko IP (0–1), makin tinggi makin berisiko.
device_trust_score	Skor kepercayaan perangkat (0–1), makin tinggi makin aman.
shared_ip_users	Jumlah pengguna lain yang memakai IP yang sama.
shared_device_users	Jumlah pengguna lain yang pernah memakai perangkat fisik yang sama.
account_age_days	Lama umur akun (hari). Akun baru → nilai kecil.
has_chargeback_history	Riwayat chargeback (0 = tidak, 1 = ya).
merchant_risk	Skor risiko merchant berdasarkan kategori.
country_risk	Skor risiko negara tempat transaksi.
distance_from_home	Jarak lokasi transaksi dari rumah pengguna.
is_new_country	Apakah ini pertama kali user bertransaksi di negara tersebut (1 = ya, 0 = tidak).
is_fraud	Label transaksi: 0 = normal, 1 = fraud.

BAB IV
EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis merupakan tahap awal yang bertujuan untuk memahami karakteristik data secara menyeluruh sebelum penerapan algoritma pembelajaran mesin, sehingga keputusan pada tahap cleaning, transformasi, dan pemodelan memiliki dasar empiris yang kuat. EDA berfungsi untuk memastikan dataset sintetis fraud yang digunakan benar-benar dipahami dari sisi struktur, tipe fitur, distribusi nilai, serta hubungannya dengan label. Hasil EDA pada bab ini menjadi landasan langsung bagi implementasi Decision Tree Learning, Logistic Regression, dan KNN baik from scratch maupun menggunakan pustaka.

4.1 Data size

Menilai kualitas, struktur, dan kelengkapan data agar dapat dipastikan bahwa dataset layak digunakan untuk proses pembelajaran mesin sesuai spesifikasi tugas. Pemahaman ini mencakup ukuran data, pengelompokan fitur, tipe data, sebaran nilai numerik, serta indikasi hubungan awal antara fitur dan label fraud. Seluruh analisis dilakukan secara sistematis agar hasilnya dapat ditelusuri kembali dan direproduksi.

4.1.1 The size of the data

Mengidentifikasi skala dataset yang digunakan sehingga dapat menilai kompleksitas komputasi dan kecukupan data untuk pelatihan model pembelajaran mesin. Dataset yang digunakan memiliki ukuran 100000 baris dan 31 kolom yang menunjukkan bahwa data cukup besar untuk menangkap pola fraud namun tetap memungkinkan pelatihan model berbasis aturan dan jarak tanpa reduksi dimensi awal. Berikut kode yang digunakan untuk menampilkan ukuran dataset.

```
print(f"Size of the data: {train.shape} \n")
```

4.1.2 Categorization of Features

Mengelompokkan fitur berdasarkan peran dan sifat statistiknya agar setiap algoritma menerima tipe input yang sesuai. Dari total 31 kolom, terdapat 3 kolom identifier yaitu ID, transaction_id, dan user_id yang tidak mengandung informasi prediktif langsung sehingga dikeluarkan dari pemodelan.

Hasil klasifikasi menunjukkan terdapat 3 fitur biner yaitu gender, has_chargeback_history, dan is_new_country yang memiliki dua nilai unik, 6 fitur nominal seperti country, device_type, device_os, merchant_category, transaction_type, dan day_of_week yang merepresentasikan kategori tanpa urutan numerik, serta 18 fitur numerik seperti transaction_amount, account_age_days, dan distance_from_home yang memiliki rentang nilai kontinu dan variasi tinggi. Berikut kode yang digunakan untuk mengelompokkan fitur berdasarkan jumlah nilai unik dan tipe data.

```
def classify_features(df, id_cols=['ID', 'transaction_id',
'user_id'], target_col='is_fraud'):
    # Inisialisasi List
    cols_drop = id_cols.copy()
    cols_binary = []
    cols_nominal = []
    cols_numerical = []

    # Iterasi setiap kolom
    for col in df.columns:
        if col in id_cols or col == target_col:
            continue

        n_unique = df[col].nunique()
        dtype = df[col].dtype

        # Kolom binary
        if n_unique == 2:
            cols_binary.append(col)

        # Kolom nominal
        elif dtype == 'object' or
(str(dtype).startswith('int') and n_unique <= 10 and 'risk'
not in col and 'users' not in col and 'attempts' not in col):
            cols_nominal.append(col)
```

```
        # Kolom numerik
        else:
            cols_numerical.append(col)

    # Kolom sisanya didrop
    return cols_drop, cols_binary, cols_nominal,
cols_numerical

# Eksekusi
drop_list, bin_list, nom_list, num_list =
classify_features(train)

print(f"Drop/IDs: {len(drop_list)} -> {drop_list}")
print(f"Binary:   {len(bin_list)} -> {bin_list}")
print(f"Nominal:  {len(nom_list)} -> {nom_list}")
print(f"Numerical:{len(num_list)} -> {num_list}")
```

4.1.3 Data type and number of unique values

Menilai granularitas informasi pada setiap fitur serta mengidentifikasi fitur dengan variasi tinggi yang berpotensi memengaruhi pemodelan. Fitur kategorikal seperti gender memiliki 2 nilai unik yaitu M dan F, sementara device_os memiliki 5 nilai unik seperti Android, iOS, Windows, Linux, dan MacOS yang menunjukkan variasi perangkat yang cukup beragam.

Pada fitur numerik, transaction_amount memiliki 28920 nilai unik dan distance_from_home memiliki 30865 nilai unik yang menunjukkan distribusi kontinu dengan resolusi tinggi, sedangkan fitur seperti failed_login_attempts hanya memiliki 6 nilai unik yang menunjukkan sifat diskret dengan batasan kecil.

Tabel 2 Data Type, Number, and Sample of Unique Values

Column	Dtype	Unique Count	Unique Values Sample
gender	object	2	['M', 'F']
has_chargeback_history	int64	2	[0, 1]
is_new_country	int64	2	[0, 1]
country	object	10	['UK', 'IN', 'US', 'ID', 'BR', 'CA', 'DE', 'AU...

device_type	object	3	['mobile', 'desktop', 'tablet']
device_os	object	5	['Android', 'iOS', 'Windows', 'Linux', 'MacOS']
merchant_category	object	9	['clothing', 'restaurants', 'electronics', 'ga...
transaction_type	object	4	['purchase', 'topup', 'withdrawal', 'transfer']
day_of_week	int64	7	[1, 3, 5, 0, 4, 6, 2]

Pada fitur numerik, transaction_amount memiliki 28920 nilai unik dan distance_from_home memiliki 30865 nilai unik yang menunjukkan distribusi kontinu dengan resolusi tinggi, sedangkan fitur seperti failed_login_attempts hanya memiliki 6 nilai unik yang menunjukkan sifat diskret dengan batasan kecil.

Tabel 3 Data Type, Number of Unique Values

Column	Dtype	Unique Count
age	int64	62
transaction_amount	float64	28920
time_of_day	int64	24
transaction_duration	float64	2448
num_prev_transactions	int64	49
avg_transaction_amount	float64	14709
std_transaction_amount	float64	7501
transactions_last_24h	int64	45
transactions_last_1h	int64	16
failed_login_attempts	int64	6
ip_risk_score	float64	1001
device_trust_score	float64	1001
account_age_days	int64	2999
shared_ip_users	int64	12
shared_device_users	int64	11
merchant_risk	float64	8

country_risk	float64	7
distance_from_home	float64	30865

Berikut kode yang digunakan untuk menghasilkan ringkasan tipe data dan jumlah nilai unik pada fitur kategorikal dan numerik.

```
# Gabungkan daftar kolom
cols_with_unique_list = bin_list + nom_list
cols_only_count = num_list

# Kolom kategorikal (binary dan nominal)
df_cat = pd.DataFrame({
    'Column': cols_with_unique_list,
    'Dtype': [train[col].dtype for col in
cols_with_unique_list],
    'Unique Count': [train[col].nunique() for col in
cols_with_unique_list],
    'Unique Values Sample':
[str(train[col].unique().tolist()[:10]) for col in
cols_with_unique_list] # dibatesin 10 aja biar rapi
})

# Kolom numerik (kontinu)
df_num = pd.DataFrame({
    'Column': cols_only_count,
    'Dtype': [train[col].dtype for col in cols_only_count],
    'Unique Count': [train[col].nunique() for col in
cols_only_count]
})

display(Markdown("\n### Kolom Kategorikal (Binary dan
Nominal)"))
display(df_cat)
display(Markdown("\n### Kolom Numerik (Kontinu)"))
display(df_num)
```

4.1.4 Values for numerical features

Memahami sebaran nilai, titik pusat, dan variasi dari setiap fitur numerik yang digunakan dalam pemodelan. Fitur age memiliki nilai minimum 18 dan maksimum 79 dengan rata-rata 48.5 dan

deviasi standar 17.9 yang menunjukkan distribusi usia yang lebar namun terpusat di usia produktif.

Fitur transaction_amount memiliki rentang dari 0 hingga 20601.1 dengan median 64.5 dan deviasi standar 376.0 yang mengindikasikan adanya transaksi bernilai sangat tinggi yang berpotensi menjadi outlier penting dalam deteksi fraud, sementara distance_from_home memiliki nilai maksimum 6388.5 dengan rata-rata 117.4 yang menunjukkan sebagian kecil transaksi dilakukan jauh dari lokasi asal pengguna.

Tabel 4 Values for Numerical Features

Column	Min	Max	Mean	Median	Std
age	18.0	79.0	48.5	48.0	17.9
transaction_amount	0.0	20601.1	126.9	64.5	376.0
time_of_day	0.0	23.0	12.8	12.0	5.7
transaction_duration	0.0	52.1	4.0	2.8	4.0
num_prev_transactions	8.0	59.0	30.0	30.0	5.5
avg_transaction_amount	-52.9	195.8	70.0	70.0	30.0
std_transaction_amount	5.0	80.0	42.5	42.5	21.7
transactions_last_24h	0.0	44.0	2.0	1.0	4.0
transactions_last_1h	0.0	15.0	0.3	0.0	1.5
failed_login_attempts	0.0	5.0	0.3	0.0	0.5
ip_risk_score	0.0	1.0	0.5	0.5	0.3
device_trust_score	0.0	1.0	0.5	0.5	0.3
account_age_days	1.0	2999.0	1497.3	1496.0	866.0
shared_ip_users	0.0	11.0	2.0	2.0	1.4
shared_device_users	0.0	11.0	1.5	1.0	1.2
merchant_risk	0.0	0.6	0.2	0.1	0.1
country_risk	0.0	0.1	0.1	0.0	0.0
distance_from_home	0.0	6388.5	117.4	71.8	212.0

Berikut kode yang digunakan untuk menghitung statistik minimum, maksimum, mean, median, dan deviasi standar setiap fitur numerik.

```
descriptive_stats    =    train[num_list].describe().T[['min',
'max', 'mean', '50%', 'std']].rename(
      columns={'50%': 'Median', 'min': 'Min', 'max': 'Max',
'mean': 'Mean', 'std': 'Std'}
).round(1).reset_index().rename(columns={'index': 'Column'})

descriptive_stats = descriptive_stats[['Column', 'Min', 'Max',
'Mean', 'Median', 'Std']]
display(descriptive_stats.style.format("{:.1f}",
subset=['Min', 'Max', 'Mean', 'Median', 'Std'])
        .set_properties(**{'text-align': 'center'})
              .set_table_styles([{'selector': 'th', 'props':
[('text-align', 'center')]}])
        .hide(axis="index"))
```

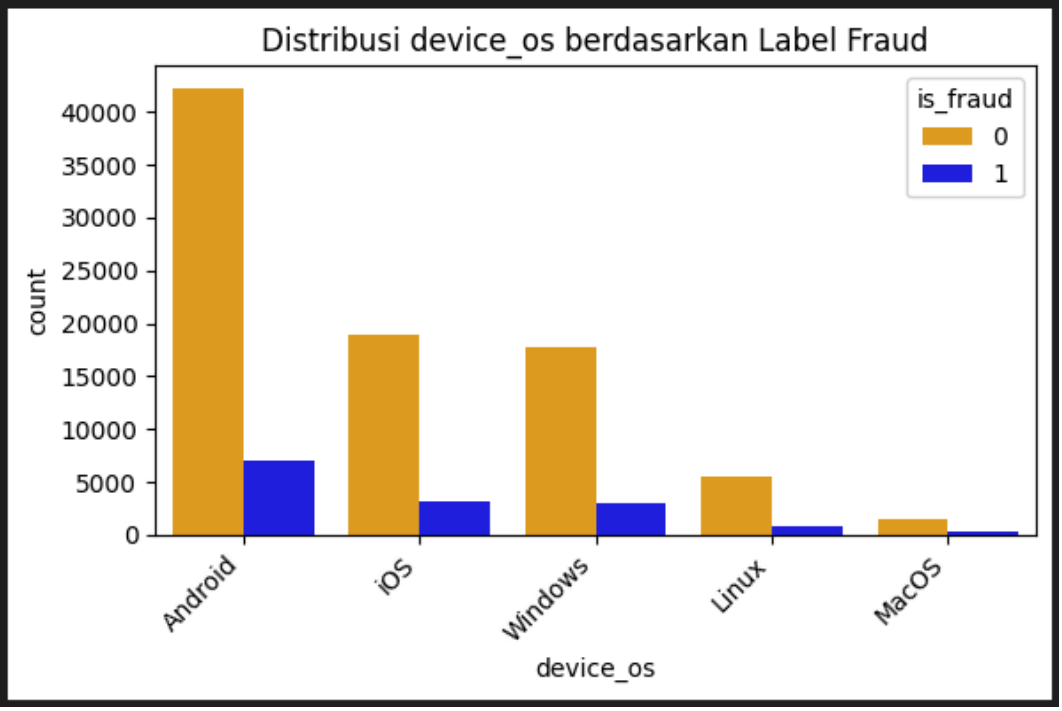
4.1.5 Correlation between categorical and label

Mengamati distribusi label is_fraud terhadap setiap kategori untuk mengidentifikasi pola perbedaan proporsi fraud. Pada fitur device_os, jumlah transaksi non-fraud tertinggi berasal dari Android sebanyak 42247 sementara transaksi fraud pada Android mencapai 6936, lebih tinggi dibandingkan sistem operasi lain seperti MacOS yang hanya memiliki 250 transaksi fraud.

Distribusi ini menunjukkan bahwa beberapa kategori tidak hanya dominan secara jumlah tetapi juga memiliki kontribusi signifikan terhadap kasus fraud, sehingga fitur kategorikal memiliki peran penting dalam pemisahan kelas pada model DTL.

Crosstab: device_os vs is_fraud

is_fraud	Android	Linux	MacOS	Windows	iOS
0	42247	5431	1513	17802	18879
1	6936	883	250	2948	3111



Berikut kode yang digunakan untuk menghasilkan tabel crosstab dan visualisasi distribusi kategori terhadap label is_fraud.

```
df_cat = train.copy()
label_col = "is_fraud"

categorical_columns = bin_list + nom_list
def analyze_and_plot_categorical(df, cols, label_col):
    display(Markdown(f"### Menganalisis **{len(cols)}** Kolom
Kategorikal"))
    for col in cols:
        if col == label_col:
            continue

        # Hitung Crosstab (Kategori vs Label)
        pair_counts = df.groupby([label_col,
```

```
col]).size().unstack(fill_value=0)

        # Cetak Hasil Crosstab
        display(Markdown(f"\n#### Crosstab: {col} vs
{label_col}"))
        print(pair_counts)

        # Plot Distribusi
        plt.figure(figsize=(6, 4))
        sns.countplot(
            data=df,
            x=col,
            hue=label_col,
            palette={0: "orange", 1: "blue"}
        )
        plt.title(f"Distribusi {col} berdasarkan Label Fraud")
        plt.xticks(rotation=45, ha="right")
        plt.tight_layout()
        plt.show()

analyze_and_plot_categorical(df_cat, categorical_columns,
label_col)
```

4.1.6 Correlation between numerical and label

Mengidentifikasi fitur numerik yang memiliki variasi tinggi dan berpotensi memisahkan kelas fraud dan non-fraud. Fitur seperti transaction_amount, ip_risk_score, dan device_trust_score memiliki rentang nilai kontinu yang luas dan jumlah nilai unik tinggi sehingga memungkinkan pemisahan berbasis threshold pada algoritma Decision Tree.

Selain itu, jumlah transaksi dalam 1 jam memiliki median 0 dan maksimum 15 yang menunjukkan bahwa lonjakan aktivitas dalam waktu singkat dapat menjadi indikator penting dalam klasifikasi fraud, terutama ketika dikombinasikan dengan fitur risiko lain. Berikut kode yang digunakan untuk menghitung jumlah nilai unik pada setiap fitur numerik serta menampilkan distribusi nilai pada fitur biner dan nominal.

```
analyze_cols = bin_list + nom_list

display(Markdown(("\n#  Value  Counts  untuk  Kolom  Biner  dan
```

```
Nominal")))
for col in analyze_cols:
    if col in train.columns:
            display(Markdown((f"\n#####  ==========  {col}  vs
{label_col} ==========")))
        print(train[col].value_counts(dropna=False))
    else:
           print(f"[WARNING]  Kolom  '{col}'  tidak  ditemukan  di
DataFrame train.")

display(Markdown(("\n#  Jumlah  Unique  Values  untuk  Kolom
Numerik Kontinu")))
for col in num_list:
    if col in train.columns:
        print(f"{col}: {train[col].nunique()}")
```

4.2 Missing values for each feature

Mengukur kelengkapan data pada setiap fitur agar tahap Data Cleaning dan Data Transformation pada spesifikasi tugas dapat dilakukan berdasarkan angka yang jelas. Hasil ini dipakai untuk menentukan fitur mana yang harus diimputasi dan fitur mana yang aman dipakai tanpa perlakuan khusus sebelum melatih DTL dan KNN from scratch.

4.2.1 Missing values for each feature

Menghitung jumlah nilai hilang pada setiap kolom sehingga diketahui fitur mana yang mengandung NaN dan berapa banyak baris yang terdampak. Dataset memiliki missing values hanya pada tiga fitur numerik yaitu transaction_amount sebanyak 2419, avg_transaction_amount sebanyak 2419, dan device_trust_score sebanyak 2419 sementara seluruh kolom lain termasuk is_fraud memiliki missing 0 sehingga masalah kelengkapan terpusat pada tiga fitur tersebut. Berikut kode yang digunakan untuk menghitung missing values per fitur pada dataset train.

```
def missing_value(df):
    df = df.drop(columns=['id'], errors='ignore')
    return df.isna().sum().reset_index(name='missing')

display(missing_value(train))
```

4.2.2 Outliers for each feature

Menghitung jumlah outlier pada setiap fitur numerik menggunakan aturan IQR sehingga dapat terlihat fitur mana yang memiliki nilai ekstrem yang berpotensi memengaruhi pembentukan threshold pada DTL dan perhitungan jarak pada KNN. Hasil perhitungan menunjukkan outlier tertinggi ada pada transactions_last_1h sebanyak 11697 dan transactions_last_24h sebanyak 8878, lalu transaction_amount sebanyak 5904 dan distance_from_home sebanyak 5550 yang konsisten dengan rentang maksimum besar pada statistik sebelumnya serta mengindikasikan adanya aktivitas transaksi padat dan jarak transaksi yang sangat jauh pada sebagian data. Berikut kode yang digunakan untuk menghitung jumlah outlier tiap fitur numerik dengan batas bawah dan atas berbasis kuartil Q1, Q3, dan IQR.

```
outlier_df = pd.DataFrame({
      col: ((train[col] < train[col].quantile(0.25) - 1.5 *
(train[col].quantile(0.75) - train[col].quantile(0.25))) |
             (train[col] > train[col].quantile(0.75) + 1.5 *
(train[col].quantile(0.75)                                  -
train[col].quantile(0.25)))).sum()
    for col in num_list
}, index=['Outlier Count']).T

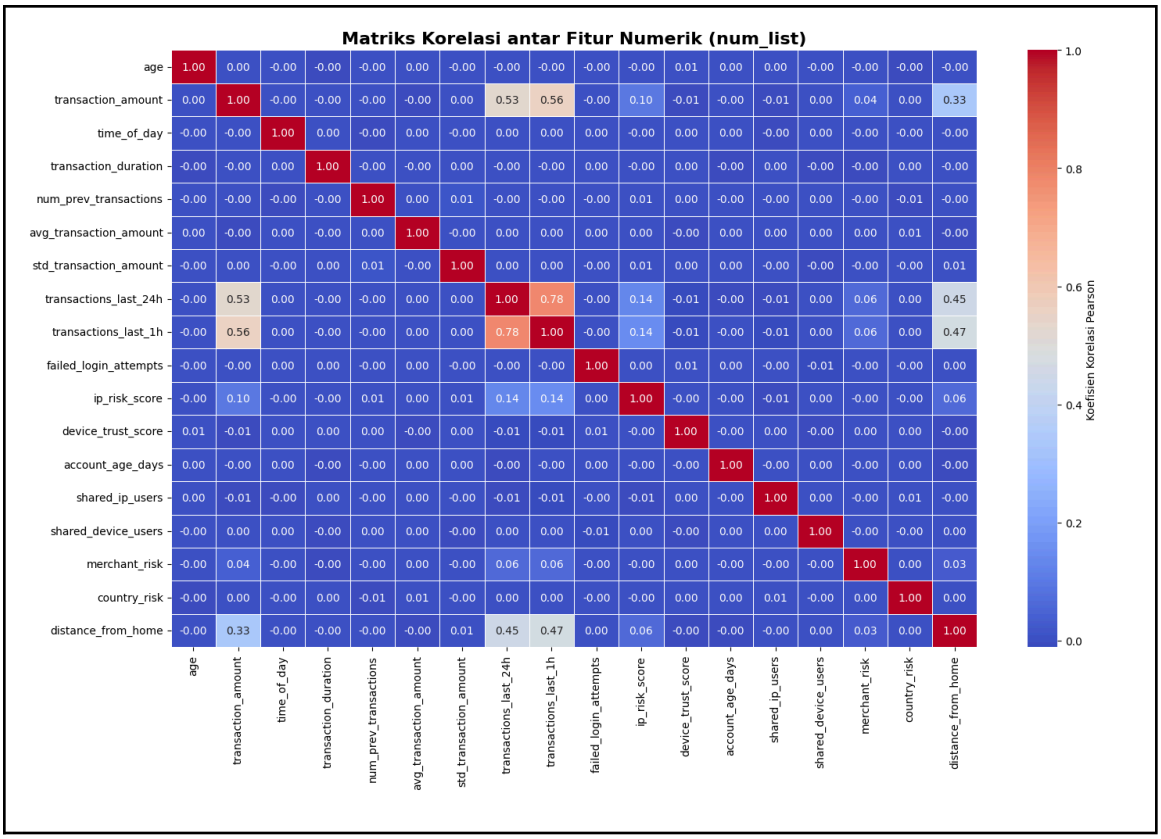
display(outlier_df)
```

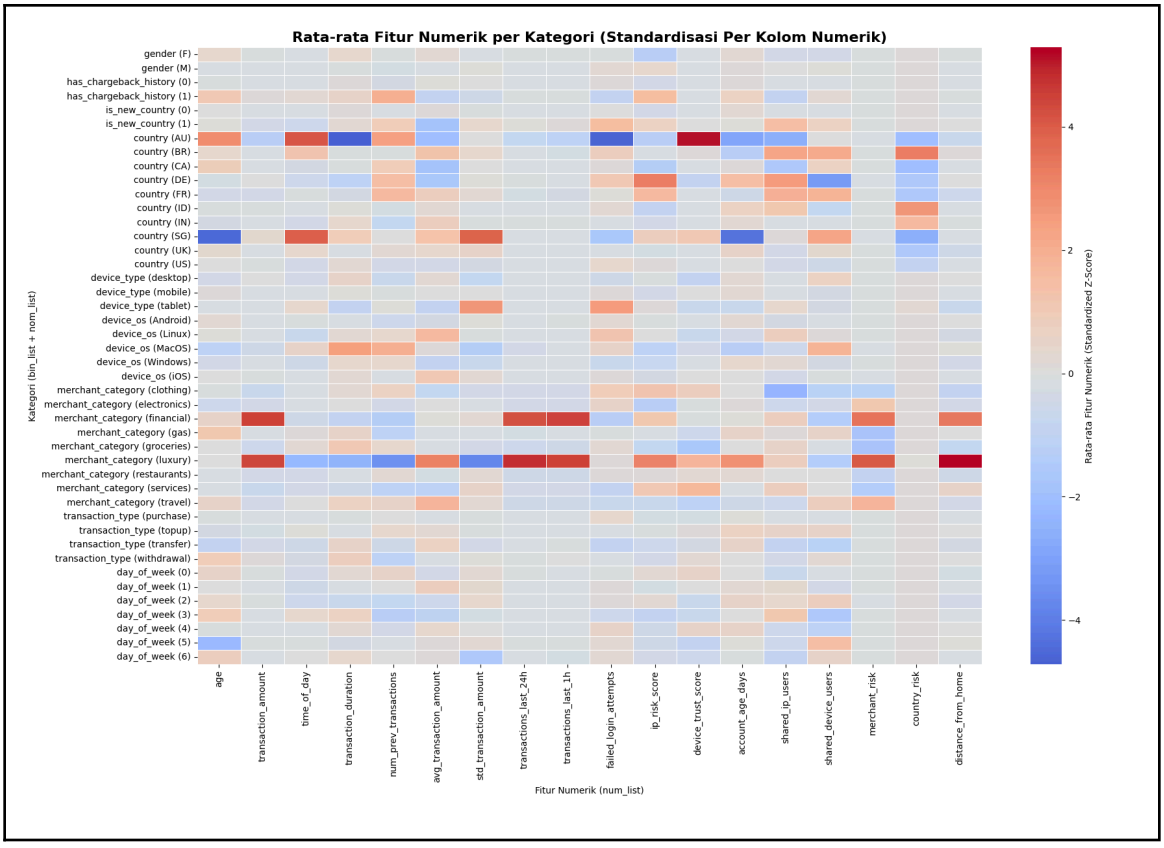
4.2.3 Correlations between features

Melihat hubungan antar fitur untuk mengidentifikasi pasangan fitur yang bergerak bersama, fitur yang saling tumpang tindih informasi, serta indikasi keterkaitan yang dapat memengaruhi pemilihan fitur dan perilaku split pada pohon keputusan. Analisis dilakukan dalam dua sudut pandang yaitu korelasi numerik dengan numerik menggunakan Pearson dan perbandingan numerik terhadap kategori dengan menghitung rata-rata setiap fitur numerik per kategori lalu distandardisasi menjadi Z-score sehingga pola per kategori dapat dibandingkan pada skala yang sama.

```

```


Matriks Korelasi antar Fitur Numerik (num_list)
age 1.00 0.00 -0.00 -0.00 -0.00 0.00 -0.00 -0.00 -0.00 -0.00 -0.00 0.01 0.00 0.00 -0.00 -0.00 -0.00 -0.00
transaction_amount 0.00 1.00 -0.00 -0.00 -0.00 -0.00 0.00 0.53 0.56 -0.00 0.10 -0.01 -0.00 -0.01 0.00 0.04 0.00 0.33
time_of_day -0.00 -0.00 1.00 0.00 -0.00 0.00 -0.00 0.00 0.00 0.00 -0.00 0.00 0.00 -0.00 0.00 -0.00 -0.00 -0.00
transaction_duration -0.00 -0.00 0.00 1.00 -0.00 -0.00 0.00 -0.00 -0.00 0.00 -0.00 0.00 -0.00 0.00 -0.00 -0.00 0.00 -0.00
num_prev_transactions -0.00 -0.00 -0.00 -0.00 1.00 0.00 0.01 -0.00 -0.00 -0.00 0.01 0.00 -0.00 -0.00 0.00 -0.00 -0.01 -0.00
avg_transaction_amount 0.00 -0.00 0.00 -0.00 0.00 1.00 -0.00 0.00 0.00 0.00 0.00 -0.00 0.00 0.00 0.00 0.00 0.01 -0.00
std_transaction_amount -0.00 0.00 -0.00 0.00 0.01 -0.00 1.00 0.00 0.00 -0.00 0.01 0.00 0.00 -0.00 -0.00 -0.00 -0.00 0.01
transactions_last_24h -0.00 0.53 0.00 -0.00 -0.00 0.00 0.00 1.00 0.78 -0.00 0.14 -0.01 -0.00 -0.01 0.00 0.06 0.00 0.45
transactions_last_1h -0.00 0.56 0.00 -0.00 -0.00 0.00 0.00 0.78 1.00 -0.00 0.14 -0.01 -0.00 -0.01 0.00 0.06 0.00 0.47
failed_login_attempts -0.00 -0.00 0.00 0.00 -0.00 0.00 -0.00 -0.00 -0.00 1.00 0.00 0.01 0.00 -0.00 -0.01 -0.00 -0.00 0.00
ip_risk_score -0.00 0.10 -0.00 -0.00 0.01 0.00 0.01 0.14 0.14 0.00 1.00 -0.00 -0.00 -0.01 0.00 -0.00 -0.00 0.06
device_trust_score 0.01 -0.01 0.00 0.00 0.00 -0.00 0.00 -0.01 -0.01 0.01 -0.00 1.00 -0.00 0.00 -0.00 0.00 0.00 -0.00
account_age_days 0.00 -0.00 0.00 -0.00 -0.00 0.00 0.00 -0.00 -0.00 0.00 -0.00 -0.00 1.00 -0.00 0.00 -0.00 0.00 -0.00
shared_ip_users 0.00 -0.01 -0.00 0.00 -0.00 0.00 -0.00 -0.01 -0.01 -0.00 -0.01 0.00 -0.00 1.00 0.00 -0.00 0.01 -0.00
shared_device_users -0.00 0.00 0.00 -0.00 0.00 0.00 -0.00 0.00 0.00 -0.01 0.00 -0.00 0.00 0.00 1.00 -0.00 -0.00 0.00
merchant_risk -0.00 0.04 -0.00 -0.00 -0.00 0.00 -0.00 0.06 0.06 -0.00 -0.00 0.00 -0.00 -0.00 -0.00 1.00 0.00 0.03
country_risk -0.00 0.00 -0.00 0.00 -0.01 0.01 -0.00 0.00 0.00 -0.00 -0.00 0.00 0.00 0.01 -0.00 0.00 1.00 0.00
distance_from_home -0.00 0.33 -0.00 -0.00 -0.00 -0.00 0.01 0.45 0.47 0.00 0.06 -0.00 -0.00 -0.00 0.00 0.03 0.00 1.00
age transaction_amount time_of_day transaction_duration num_prev_transactions avg_transaction_amount std_transaction_amount transactions_last_24h transactions_last_1h failed_login_attempts ip_risk_score device_trust_score account_age_days shared_ip_users shared_device_users merchant_risk country_risk distance_from_home
Koefisien Korelasi Pearson
1.0
0.8
0.6
0.4
0.2
0.0


Rata-rata Fitur Numerik per Kategori (Standardisasi Per Kolom Numerik)
Kategori (bin_list + nom_list)
gender (F)
gender (M)
has_chargeback_history (0)
has_chargeback_history (1)
is_new_country (0)
is_new_country (1)
country (AU)
country (BR)
country (CA)
country (DE)
country (FR)
country (ID)
country (IN)
country (SG)
country (UK)
country (US)
device_type (desktop)
device_type (mobile)
device_type (tablet)
device_os (Android)
device_os (Linux)
device_os (MacOS)
device_os (Windows)
device_os (iOS)
merchant_category (clothing)
merchant_category (electronics)
merchant_category (financial)
merchant_category (gas)
merchant_category (groceries)
merchant_category (luxury)
merchant_category (restaurants)
merchant_category (services)
merchant_category (travel)
transaction_type (purchase)
transaction_type (topup)
transaction_type (transfer)
transaction_type (withdrawal)
day_of_week (0)
day_of_week (1)
day_of_week (2)
day_of_week (3)
day_of_week (4)
day_of_week (5)
day_of_week (6)
age transaction_amount time_of_day transaction_duration num_prev_transactions avg_transaction_amount std_transaction_amount transactions_last_24h transactions_last_1h failed_login_attempts ip_risk_score device_trust_score account_age_days shared_ip_users shared_device_users merchant_risk country_risk distance_from_home
Fitur Numerik (num_list)
Rata-rata Fitur Numerik (Standardized Z-Score)
4
2
0
−2
−4

Berikut kode yang digunakan untuk menghitung korelasi Pearson pada fitur numerik dan membangun heatmap perbandingan kategorikal-numerik melalui mean per kategori dan standardisasi Z-score.

```
def numeric_correlations_simplified(df, num_list):
    # Hitung Matriks Korelasi Pearson
    corr_matrix = df[num_list].corr()

    # Visualisasi
    plt.figure(figsize=(15, 10))
     sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f',
linewidths=0.5,
                cbar_kws={'label': 'Koefisien Korelasi Pearson'})
      plt.title('Matriks Korelasi antar Fitur Numerik (num_list)',
fontsize=16, fontweight='bold')
    plt.tight_layout()
    plt.show()

def      categorical_numeric_comparison_simplified(df,      cat_list,
num_list):
    all_cat_cols = cat_list
    heatmap_data = {}

    # Mean Per Kategori
    for cat_col in all_cat_cols:
        grouped_mean = df.groupby(cat_col)[num_list].mean()
        for cat_val in grouped_mean.index:
            col_name = f"{cat_col} ({cat_val})"
            heatmap_data[col_name] = grouped_mean.loc[cat_val]

    comparison_df = pd.DataFrame(heatmap_data).T
    scaler = StandardScaler()
                                    normalized_data         =
scaler.fit_transform(comparison_df[num_list].fillna(0))
    comparison_normalized_df = pd.DataFrame(normalized_data,

index=comparison_df.index,

columns=comparison_df.columns)

    # Visualisasi Heatmap
    plt.figure(figsize=(18, 12))
    sns.heatmap(comparison_normalized_df,
```

```
                annot=False,
                cmap='coolwarm',
                fmt='.2f',
                linewidths=0.5,
                center=0,
                   cbar_kws={'label': 'Rata-rata Fitur Numerik
(Standardized Z-Score)'})

          plt.title('Rata-rata  Fitur  Numerik  per  Kategori
(Standardisasi    Per    Kolom    Numerik)',    fontsize=16,
fontweight='bold')
    plt.xlabel('Fitur Numerik (num_list)')
    plt.ylabel('Kategori (bin_list + nom_list)')
    plt.tight_layout()
    plt.show()

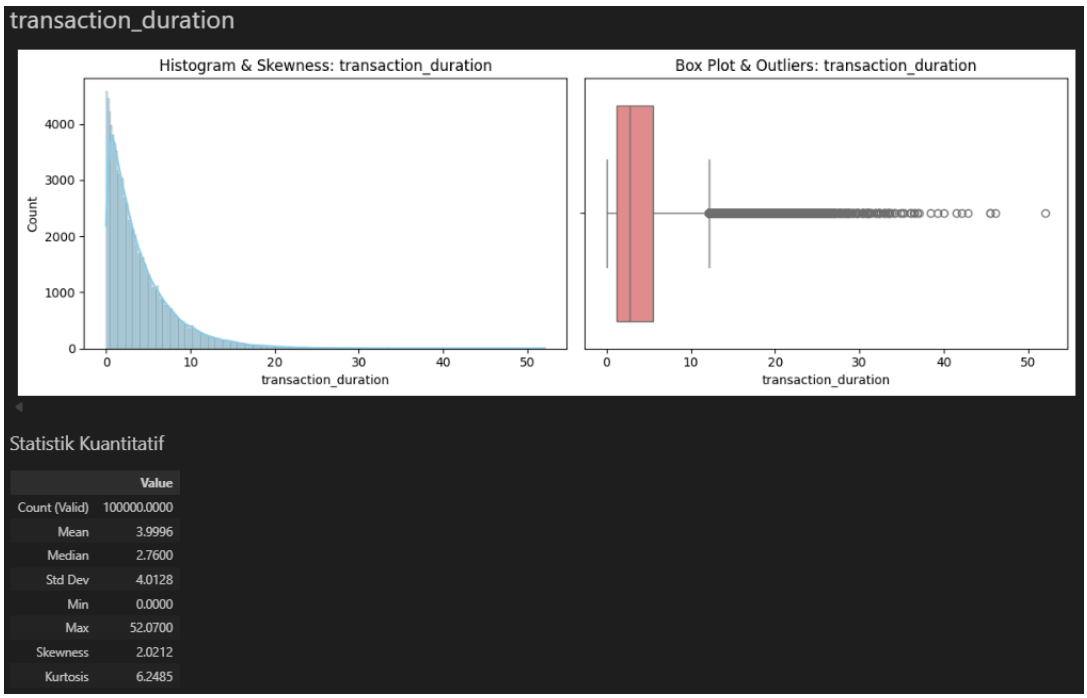
    return comparison_df

# Korelasi Numerik-Numerik
numeric_correlations_simplified(train, num_list)

# Korelasi Kategorikal-Numerik
all_categorical_cols = bin_list + nom_list
correlation_df_comparison                                    =
categorical_numeric_comparison_simplified(train,
all_categorical_cols, num_list)
```

4.2.4 Distribution of each feature

Memeriksa bentuk distribusi setiap fitur numerik agar terlihat apakah data simetris, miring, memiliki puncak tajam, atau memiliki ekor panjang, sekaligus memvalidasi keberadaan outlier secara visual. Untuk setiap fitur dalam num_list, ditampilkan histogram dengan KDE untuk melihat bentuk sebaran dan box plot untuk melihat posisi median serta nilai ekstrem, lalu dihitung statistik kuantitatif yang memuat jumlah data valid, mean, median, standar deviasi, nilai minimum, maksimum, skewness, dan kurtosis agar bentuk distribusi dapat dijelaskan dengan angka yang terukur.



Berikut kode yang digunakan untuk membuat histogram, box plot, serta menghitung skewness dan kurtosis pada setiap fitur numerik setelah menghapus NaN.

```
for col in num_list:
    if col == "id": continue

    series = train[col].dropna()

    if series.empty:
        display(Markdown(f"## {col} (Data Kosong/NaN)\n"))
            print("Kolom ini tidak memiliki data valid untuk
dianalisis.")
        continue

    # Print Judul
    display(Markdown(f"## {col}\n"))

    # Visualisasi skewness
    plt.figure(figsize=(12, 4))

    # Histogram
    plt.subplot(1, 2, 1)
    sns.histplot(series, kde=True, color='skyblue')
```

```
    plt.title(f'Histogram & Skewness: {col}')

    # Box Plot
    plt.subplot(1, 2, 2)
    sns.boxplot(x=series, color='lightcoral')
    plt.title(f'Box Plot & Outliers: {col}')

    plt.tight_layout()
    plt.show()

    # Statistik fitur kuantitatif
    skewness = skew(series)
    kurt = kurtosis(series)

    stats_data = {
        'Count (Valid)': [len(series)],
        'Mean': [series.mean()],
        'Median': [series.median()],
        'Std Dev': [series.std()],
        'Min': [series.min()],
        'Max': [series.max()],
        'Skewness': [skewness],
        'Kurtosis': [kurt],
    }
      stats_df = pd.DataFrame(stats_data).T.rename(columns={0:
'Value'})
    display(Markdown("### Statistik Kuantitatif"))

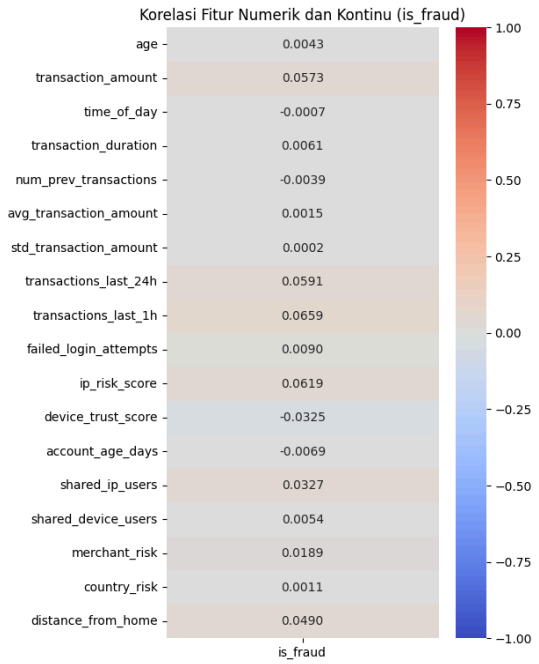
    # Lakukan rounding pada kolom 'Value'
    stats_df_rounded = stats_df.copy()
     stats_df_rounded.loc[['Mean', 'Median', 'Std Dev', 'Min',
'Max', 'Skewness', 'Kurtosis'], 'Value'] = \
           stats_df_rounded.loc[['Mean', 'Median', 'Std Dev',
'Min', 'Max', 'Skewness', 'Kurtosis'], 'Value'].round(4)

    display(stats_df_rounded)
```

4.2.5 Correlation between features and the target variable

Mengukur keterkaitan linear antara setiap fitur numerik dengan label is_fraud agar terlihat fitur mana yang memiliki sinyal langsung terhadap target pada level korelasi Pearson. Proses ini menghitung matriks korelasi untuk seluruh fitur numerik dan mengambil hanya kolom korelasi

terhadap is_fraud, lalu divisualisasikan dalam heatmap dengan rentang nilai dari -1 sampai 1 dan angka korelasi ditampilkan hingga 4 desimal agar perbedaan kecil antar fitur tetap terbaca.



Berikut kode yang digunakan untuk menghitung korelasi Pearson fitur numerik terhadap is_fraud dan memvisualisasikannya dalam heatmap satu kolom.

```
target_col = 'is_fraud'
correlation_columns = num_list + [target_col]

# Hitung Matriks Korelasi
corr_matrix = train[correlation_columns].corr()

# 2. Ambil Korelasi Fitur vs is fraud
corr_feature_target                                          =
corr_matrix[[target_col]].drop(index=target_col)

# 3. Plotting Heatmap
plt.figure(figsize=(5,    max(5,    len(corr_feature_target)    *
0.5)))
sns.heatmap(corr_feature_target,  annot=True,  cmap='coolwarm',
vmin=-1, vmax=1, fmt='.4f')
plt.title(f'Korelasi     Fitur     Numerik     dan     Kontinu
({target_col})')
plt.show()
```

4.3 Split Training Set, Validation Set, Testing Set

Pembagian data ke dalam *training set* dan *validation set* berfungsi sebagai langkah awal untuk mengevaluasi kinerja model yang akan dilatih. Proses pembagian ini dilakukan sebelum tahap *preprocessing* untuk menghindari terjadinya *data leakage* antar himpunan data. Dengan melakukan pemisahan sejak awal, informasi dari data validasi tidak akan memengaruhi proses pembelajaran model, sehingga evaluasi yang dilakukan dapat memberikan kinerja model secara lebih objektif.

```
original_train_set = train.copy()
# Rasio validation size, yaitu 20%
val_size = 0.2
random_state = 31

# Split jadi train_set dan val_set
train_set, val_set = train_test_split(
    train,
    test_size=val_size,
    random_state=random_state,
    shuffle=True
)

print("Train size:", len(train_set))
print("Validation size:", len(val_set))
```

BAB V
CLEANING DAN PREPROCESSING DATA

Tahap *data cleaning* dan *preprocessing* merupakan langkah awal yang dilakukan setelah mengetahui gambaran umum terhadap dataset yang diperoleh. Data mentah umumnya belum siap untuk langsung digunakan dalam proses pelatihan model, sehingga diperlukan berbagai tahapan untuk membersihkan dan memformat data sehingga dapat diinterpretasikan dengan baik oleh algoritma *machine learning.* Melalui proses ini, dataset dipersiapkan menjadi lebih terstruktur dan representatif, sehingga dapat menghasilkan model yang lebih akurat.

5.1 Data Cleaning

Data cleaning merupakan tahap awal yang sangat penting dalam mempersiapkan dataset untuk proses *machine learning.* Data mentah yang dikumpulkan, umumnya terdapat nilai yang hilang, kesalahan, duplikasi, dan inkonsistensi yang dapat memengaruhi kualitas analisis dan kinerja model. Maka, proses *data cleaning* dilakukan untuk memastikan integritas dan konsistensi data melalui penanganan *missing data,* identifikasi dan penanganan *outliers*, validasi data, penghapusan duplikasi, serta penerapan *feature engineering* yang sesuai, sehingga dataset yang digunakan menjadi lebih representatif dan siap untuk tahap pemodelan.

5.1.1 Handling Missing Data

Data yang hilang (*missing data)* merupakan permasalahan umum dalam pengolahan data dan dapat berdampak negatif terhadap kinerja serta akurasi model *machine learning.* Nilai yang hilang dapat menyebabkan bias pada proses pelatihan model, memengaruhi perhitungan statistik, dan memengaruhi hasil prediksi. Oleh karena itu, diperlukan strategi penanganan *missing data* yang sesuai dengan karakteristik dataset dan permasalahan yang dihadapi, baik melalui teknik imputasi, penghapusan data, maupun pendekatan berbasis pengetahuan domain, sehingga kualitas data tetap terjaga dan model yang dibangun dapat memberikan hasil yang lebih baik.

```
def impute_missing_values(df_train, df_val, num_cols, nom_cols,
bin_cols):
    train_imputed = df_train.copy()
    val_imputed = df_val.copy()

    #imputasi numerikal -> median
    for col in num_cols:
        if col in train_imputed.columns:
            median_val = train_imputed[col].median()

            train_imputed[col] =
train_imputed[col].fillna(median_val)
            val_imputed[col] =
val_imputed[col].fillna(median_val)

    #imputasi kategorikal (modus) pada gabungan nominal &
binary
    cat_cols = nom_cols + bin_cols
    for col in cat_cols:
        if col in train_imputed.columns:
            if train_imputed[col].dropna().empty: continue

            mode_val = train_imputed[col].mode()[0] #
            train_imputed[col] =
train_imputed[col].fillna(mode_val)
            val_imputed[col] =
val_imputed[col].fillna(mode_val)

    #cek sisa missing values
    remaining = train_imputed.isnull().sum().sum() +
val_imputed.isnull().sum().sum()
    print(f"Total Missing Values Tersisa: {remaining}")

    return train_imputed, val_imputed
```

Fungsi impute_missing_values pada kode tersebut digunakan untuk menangani *missing values* pada data pelatihan dan data validasi secara konsisten untuk mencegah terjadinya *data leakage*. Proses imputasi dilakukan dengan menghitung nilai statistik hanya dari data pelatihan, kemudian menerapkannya ke kedua himpunan data. Untuk fitur numerikal, nilai yang hilang diisi dengan menggunakan median karena lebih *robust* terhadap *outlier*. Sementara itu, untuk fitur kategorikal, baik nominal maupun biner, nilai

yang hilang diisi dengan menggunakan modus yang paling sering muncul pada data pelatihan. Setelah proses imputasi selesai, fungsi ini melakukan pengecekan jumlah *missing values* yang tersisa untuk memastikan seluruh nilai yang hilang telah ditangani sebelum data digunakan pada tahap pemodelan.

5.1.2 Dealing with Outliers

Outliers adalah data yang memiliki nilai yang sangat jauh berbeda dari mayoritas data lainnya, dapat jauh lebih kecil atau jauh lebih besar yang seringkali tidak mengikuti pola distribusi data tersebut. *Outliers* dapat merusak performa model sehingga diperlukan penanganan agar model dapat belajar dari pola data yang lebih representatif. Beberapa metode yang umum digunakan untuk menangani *outliers* antara lain *imputation* dengan mengganti nilai ekstrim menggunakan nilai statistik tertentu, *clipping* dengan membatasi nilai ke batas tertentu, transformasi data seperti logaritma untuk mengurangi pengaruh nilai ekstrim, serta penggunaan model yang relatif *robust* terhadap *outliers*. Pada tugas besar ini, dilakukan dua pendekatan utama yaitu transformasi logaritmik untuk mengurangi *skewness* dan IQR *clipping* untuk membatasi nilai ekstrim, seperti pada fungsi berikut:

```
def handle_outliers_and_skew(df_train, df_val, num_cols):

    # Log Transform (Semua yang skew)
    skew_vals = df_train[num_cols].skew()
    high_skew_cols = skew_vals[abs(skew_vals) >
1.0].index.tolist()

    for col in high_skew_cols:
        if (df_train[col] >= 0).all():
            df_train[col] = np.log1p(df_train[col])
            df_val[col] = np.log1p(df_val[col])

    # IQR Clipping
    for col in num_cols:
        Q1 = df_train[col].quantile(0.25)
        Q3 = df_train[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
```

```
        df_train[col] = df_train[col].clip(lower, upper)
        df_val[col] = df_val[col].clip(lower, upper)

    return df_train, df_val
```

Pertama-tama, fungsi akan mengevaluasi fitur numerik dengan distribusi yang sangat tidak simetris (*skewed*) berdasarkan nilai *skewness* pada data pelatihan, kemudian untuk fitur non-negatif dilakukan transformasi logaritmik untuk mengurangi pengaruh nilai ekstrim dan memperbaiki bentuk distribusi data. Selanjutnya, seluruh fitur numerik diproses menggunakan metode IQR *clipping* dengan menentukan batas bawah dan atasnya berdasarkan kuartil pertama dan ketiga, sehingga nilai yang berada di luar rentang akan dipotong ke batas terdekat. Proses ini diterapkan secara konsisten pada data pelatihan dan data validasi untuk menghindari terjadinya data *leakage*.

5.1.3 Remove Duplicates

Data yang duplikat dapat menyebabkan bias pada model pembelajaran mesin sehingga dapat meningkatkan risiko *overfitting* dan memperbesar biaya komputasi karena memproses data yang tidak perlu. Untuk menangani hal ini, baris data yang memiliki nilai identik akan dihapus menggunakan fungsi di bawah ini, sehingga data yang digunakan dalam proses pelatihan bersifat unik dan representatif.

```
def remove_duplicates(df_train):
    df_clean = df_train.drop_duplicates()
    return df_clean
```

5.1.4 Feature Engineering

Feature engineering adalah proses untuk memilih, memodifikasi, atau membentuk fitur agar informasi yang terkandung di dalam data dapat dimanfaatkan secara optimal oleh model pembelajaran mesin. Jadi, akan dilakukan pemilihan terhadap fitur-fitur yang relevan dan penghapusan fitur yang tidak informatif atau redundan untuk mengurangi kompleksitas. Proses ini bertujuan untuk meningkatkan kemampuan model dalam

menangkap pola penting pada data sehingga dapat memberikan hasil prediksi yang lebih akurat. Berikut adalah implementasi dari *feature engineering*:

```
def generate_new_features(df_train, df_val, num_cols):

    def apply_fe(df):
        df_new = df.copy()

        # Pakai log1p pada amount agar skalanya tidak meledak
saat dikali risk
        if 'ip_risk_score' in df_new.columns and
'transaction_amount' in df_new.columns:
            safe_amt =
df_new['transaction_amount'].clip(lower=0)
            df_new['risk_log_amount'] =
df_new['ip_risk_score'] * np.log1p(safe_amt)

        # Ratio: Amount / Avg (Absolut)
        if 'avg_transaction_amount' in df_new.columns and
'transaction_amount' in df_new.columns:
            df_new['amount_ratio'] =
df_new['transaction_amount'] /
(df_new['avg_transaction_amount'].abs() + 1e-6)

        # Hour of Day (Fraud sering terjadi di jam aneh: 2-4
pagi)
        if 'time_of_day' in df_new.columns:
            # Ubah jam menjadi siklus (Sinus/Cosinus) agar jam
23 dekat dengan jam 0
            df_new['hour_sin'] = np.sin(2 * np.pi *
df_new['time_of_day'] / 24.0)
            df_new['hour_cos'] = np.cos(2 * np.pi *
df_new['time_of_day'] / 24.0)

        return df_new

    train_eng = apply_fe(df_train)
    val_eng = apply_fe(df_val)

    new_features = [c for c in train_eng.columns if c not in
df_train.columns]
    updated_num_cols = num_cols + new_features
    return train_eng, val_eng, updated_num_cols
```

5.2 Data Preprocessing

5.2.1 Feature Scaling

Feature scaling merupakan metode yang dilakukan untuk menyamakan skala antar fitur numerik agar setiap fitur memiliki kontribusi yang seimbang dalam proses pelatihan model. Perbedaan skala yang terlalu jauh antar fiturnya akan membuat model menjadi bias terhadap fitur tertentu. Pada tugas besar ini, digunakan metode *standardization* (*Z-score scaling*) dengan mentransformasikan setiap fitur agar memiliki rata-rata yang mendekati nol dan standar deviasi mendekati satu. Berikut merupakan implementasinya:

```
def perform_scaling(df_train, df_val=None):
    train_scaled = df_train.copy()
    val_scaled = df_val.copy()

    exclude_cols = ['is_fraud', 'ID', 'transaction_id',
'user_id']
    feature_cols = [c for c in train_scaled.columns if c not
in exclude_cols]
    scaler = StandardScaler()

    # FIT hanya di Train
    scaler.fit(train_scaled[feature_cols])

    # TRANSFORM di Train dan Val
    train_scaled[feature_cols] =
scaler.transform(train_scaled[feature_cols])
    if val_scaled is not None:
        val_scaled[feature_cols] =
scaler.transform(val_scaled[feature_cols])

    return train_scaled, val_scaled
```

5.2.2 Feature Encoding

Feature encoding merupakan proses yang dilakukan untuk mengonversi fitur kategorikal menjadi representasi numerik agar dapat diproses. Di tahap ini, fitur kategorikal dibedakan menjadi fitur biner dan nominal berdasarkan hasil analisis data. Fitur biner diubah menggunakan *ordinal encoding* agar berbentuk nilai numerik sederhana,

sedangkan fitur nominal diubah menggunakan *one-hot encoding*. Berikut adalah implementasinya:

```
def perform_encoding(df_train, df_val=None):

    # Kolom terpilih (Berdasarkan EDA)
    cols_binary  = ['gender', 'has_chargeback_history',
'is_new_country']
    cols_nominal = ['country', 'device_type', 'device_os',
'merchant_category', 'transaction_type', 'day_of_week']

    train_enc = df_train.copy()
    val_enc = df_val.copy() if df_val is not None else None

    # Proses Ordinal/Binary
    ord_enc =
OrdinalEncoder(handle_unknown='use_encoded_value',
unknown_value=-1)

    train_enc[cols_binary] =
ord_enc.fit_transform(train_enc[cols_binary])
    if val_enc is not None:
        val_enc[cols_binary] =
ord_enc.transform(val_enc[cols_binary])

    # Proses One-Hot Encoding (Nominal)
    ohe = OneHotEncoder(handle_unknown='ignore',
sparse_output=False).set_output(transform='pandas')
    train_ohe = ohe.fit_transform(train_enc[cols_nominal])
    # Masukkan hasil OHE
    train_enc =
pd.concat([train_enc.drop(columns=cols_nominal), train_ohe],
axis=1)

    if val_enc is not None:
        val_ohe = ohe.transform(val_enc[cols_nominal])
        val_enc =
pd.concat([val_enc.drop(columns=cols_nominal), val_ohe],
axis=1)

    return train_enc, val_enc
```

5.2.3 Handling Imbalanced Dataset

Dataset yang digunakan pada tugas besar ini memiliki distribusi kelas yang tidak seiimbang, di mana jumlah data pada kelas mayoritas jauh lebih besar dibandingkan kelas minoritas sehingga model pembelajaran cenderung bias terhadap kelas mayoritas dan menghasilkan performa yang buruk untuk mendeteksi kelas minoritas. Untuk mengatasi permasalahan tersebut, dilakukan metode oversampling dengan Synthetic Minority Over-sampling Technique (SMOTE) yang menghasilkan sampel sintesis pada kelas minoritas berdasarkan kedekatan data, sehingga distribusi kelas menjadi lebih seimbang tanpa menghilangkan kelas mayoritas. Berikut adalah implementasi dari metode ini:

```
def perform_resampling(X_train_processed, y_train):
    smote = SMOTE(random_state=42)
    X_resampled, y_resampled =
smote.fit_resample(X_train_processed, y_train)
    return X_resampled, y_resampled
```

5.2.4 Data Normalization

Data normalization merupakan proses yang dilakukan untuk mengurangi pengaruh perbedaan skala

```
def perform_normalization(df_train, df_val, num_cols):
    train_norm = df_train.copy()
    val_norm = df_val.copy()

    # Deteksi Skewness Otomatis
    valid_cols = [c for c in num_cols if c in
train_norm.columns]
    skew_vals = train_norm[valid_cols].skew()
    skewed_cols = skew_vals[skew_vals > 1].index.tolist()

    # Log Transform
    skipped = []
    for col in skewed_cols:
        # Cek nilai negatif
        if (train_norm[col] < 0).any():
            skipped.append(col)
            continue
```

```
        train_norm[col] = np.log1p(train_norm[col])
        val_norm[col] = np.log1p(val_norm[col])

    return train_norm, val_norm
```

5.2.5 Dimensionality Reduction

Dimensionality reduction merupakan metode yang dilakukan untuk mengurangi jumlah dimensi atau fitur dalam dataset dengan tetap mempertahankan informasi penting sebanyak mungkin. Tingginya jumlah fitur, terutama setelah proses feature encoding, dapat menyebabkan *curse of dimensionality* yang menurunkan kinerja model dan meningkatkan beban komputasi secara signifikan. Pada tugas besar ini, digunakan metode *Principal Component Analysis* (PCA) yang mentransformasikan data ke dalam dimensi yang lebih rendah dengan tetap mempertahankan 95% variansi dari data aslinya. Berikut merupakan implementasinya:

```
def perform_dim_reduction(X_train, X_val, y_train_viz=None,
n_components=0.95):
    pca = PCA(n_components=n_components, random_state=42)
    X_train_pca = pca.fit_transform(X_train)
    X_val_pca = pca.transform(X_val)

    return X_train_pca, X_val_pca
```

BAB VI
IMPLEMENTASI ALGORITMA

6.1 Algoritma Decision Tree Learning

Algoritma C4.5 diimplementasikan dari awal (*from scratch*) menggunakan konsep *entropy* sebagai ukuran ketidakpastiannya dan memilih atribut berdasarkan penurunan *entropy* setelah pemisahan data. Dimana atribut dengan penurunan *entropy* terkecil yang akan dipilih sebagai atribut pemisah. Berikut merupakan implementasi dari algoritma ini menggunakan bahasa pemrograman Python:

```
# C4.5

import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score

def entropy(y):
    n = len(y)
    if n == 0:
        return 0.0

    p1 = np.mean(y == 1)
    p0 = 1.0 - p1

    eps = 1e-12
    return -(p0 * np.log2(p0 + eps) + p1 * np.log2(p1 + eps))

def entropy_split(left_y, right_y):
    n_left = len(left_y)
    n_right = len(right_y)
    n_total = n_left + n_right

    if n_left == 0 or n_right == 0:
        return 999
```

```
    e_left = entropy(left_y)
    e_right = entropy(right_y)

    return (n_left/n_total) * e_left + (n_right/n_total) * e_right

def best_split_numeric(X_col, y):
    col = X_col.dropna().values.astype(float)
    if len(col) < 2:
        return None, -999

    parent_entropy = entropy(y)

    unique_vals = np.unique(col)
    if len(unique_vals) < 50:
        thresholds = (unique_vals[:-1] + unique_vals[1:]) / 2
    else:
        thresholds = np.percentile(col, np.linspace(5, 95, 50))

    best_t = None
    best_gain = -999

    for t in thresholds:
        left_mask = X_col <= t
        right_mask = X_col > t

        if left_mask.sum() == 0 or right_mask.sum() == 0:
            continue

        y_left = y[left_mask]
        y_right = y[right_mask]

        n = len(y)
        weighted_entropy = (
            len(y_left)/n * entropy(y_left) +
            len(y_right)/n * entropy(y_right)
        )

        gain = parent_entropy - weighted_entropy
```

```
        if gain > best_gain:
            best_gain = gain
            best_t = t

    return best_t, best_gain

def best_split_categorical(X_col, y):
    categories = np.unique(X_col)
    parent_entropy = entropy(y)

    best_cat = None
    best_gain = -999

    for c in categories:
        left_mask = (X_col == c)
        right_mask = (X_col != c)

        if left_mask.sum() == 0 or right_mask.sum() == 0:
            continue

        y_left = y[left_mask]
        y_right = y[right_mask]

        n = len(y)
        weighted_entropy = (
            len(y_left)/n * entropy(y_left) +
            len(y_right)/n * entropy(y_right)
        )

        gain = parent_entropy - weighted_entropy

        if gain > best_gain:
            best_gain = gain
            best_cat = c

    return best_cat, best_gain


def find_best_split(X, y):
```

```
    best_feature = None
    best_threshold = None
    best_score = -999
    feature_type = None

    for feature in X.columns:
        col = X[feature]

        if col.dtype.kind in 'bifc':
            t, score = best_split_numeric(col, y)
            if t is not None and score > best_score:
                best_score = score
                best_threshold = t
                best_feature = feature
                feature_type = "numeric"

        else:
            c, score = best_split_categorical(col, y)
            if c is not None and score > best_score:
                best_score = score
                best_threshold = c
                best_feature = feature
                feature_type = "categorical"

    return best_feature, best_threshold, feature_type, best_score

class Node:
    def __init__(self, feature=None, threshold=None, left=None,
right=None, value=None, feature_type=None):
        self.feature = feature
        self.threshold = threshold
        self.feature_type = feature_type
        self.left = left
        self.right = right
        self.value = value

def build_tree(X, y, depth=0, max_depth=0, min_samples=0,
min_samples_leaf=0):
    if len(np.unique(y)) == 1:
```

```
        return Node(value=y.mean())

    if depth >= max_depth or len(y) < min_samples:
        return Node(value=y.mean())

    feature, threshold, ftype, g = find_best_split(X, y)

    if feature is None or g <= 0:
        return Node(value=y.mean())

    if feature is None:
        return Node(value=y.mean())

    if ftype == "numeric":
        left_mask  = X[feature] <= threshold
        right_mask = X[feature] > threshold
    else:
        left_mask  = X[feature] == threshold
        right_mask = X[feature] != threshold

    if left_mask.sum() < min_samples_leaf or right_mask.sum() <
min_samples_leaf:
        return Node(value=y.mean())

    left = build_tree(X[left_mask], y[left_mask], depth+1,
max_depth, min_samples)
    right = build_tree(X[right_mask], y[right_mask], depth+1,
max_depth, min_samples)

    return Node(feature=feature, threshold=threshold,
feature_type=ftype,
                left=left, right=right,
                value=None)

def predict_one(row, node):
    while node.value is None:
        if node.feature_type == "numeric":
            if row[node.feature] <= node.threshold:
                node = node.left
```

```
            else:
                node = node.right
        else:
            if row[node.feature] == node.threshold:
                node = node.left
            else:
                node = node.right
    return node.value

def predict_proba(X, tree):
    return np.array([predict_one(row, tree) for _, row in
X.iterrows()])

X = train.drop(columns=["is_fraud", "ID", "transaction_id",
"user_id"])
y = train["is_fraud"]

from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

tree = build_tree(X_train, y_train, max_depth=6, min_samples=1000,
min_samples_leaf=30)

y_proba = predict_proba(X_val, tree)
y_pred = (y_proba >= 0.15).astype(int)
```

Secara garis besar, implementasi ini dimulai dengan menghitung nilai *entropy* dari sekumpulan data yang kemudian akan dievaluasi setiap kemungkinan pemisahan atributnya. Lalu akan dipilih atribut dengan penurunan *entropy* terkecil sebagai pemisahnya. Selanjutnya membangun pohon keputusan secara rekursif hingga kedalaman maksimum yang telah ditentukan atau batas jumlah data terpenuhi. Berikut adalah penjelasan dari setiap fungsi yang digunakan:

a. entropy(y)

Digunakan untuk menghitung nilai *entropy* dari sekumpulan label target y. Di mana nilai ini merepresentasikan tingkat ketidakpastian atau ketidakmurnian data pada suatu *node* dengan menghitung proporsi kelas positif dan negatif dan memasukkannya ke dalam rumus. Dalam fungsi ini, ditambahkan nilai kecil *eps* untuk mencegah terjadinya kesalahan perhitungan akibat logaritma dari nol.

b. entropy_split(left_y, right_y)

   Digunakan untuk menghitung *entropy* setelah data dibagi menjadi subset kiri dan subset kanan. Nilai *entropy* dari setiap subset akan dihitung dan digabungkan menggunakan bobot berdasarkan proporsi jumlah data pada masing-masing subset. Fungsi ini akan mengevaluasi kualitas pemisah atribut dengan melihat mana atribut yang menghasilkan nilai *entropy* yang lebih kecil karena ini menunjukkan pemisahan yang lebih baik.

c. best_split_numeric(X_col, y)

   Digunakan untuk mencari *threshold* terbaik untuk atribut numerik dengan menentukan beberapa kandidat *threshold* berdasarkan nilai persentil dari data, kemudian mencoba setiap *threshold* untuk membagi data menjadi dua bagian. Di setiap kemungkinan *threshold*, fungsi akan menghitung nilai *entropy* setelah pemisahan dan memilih *threshold* dengan *entropy* terendah sebagai pemisahan terbaik.

d. best_split_categorical(X_col, y)

   Digunakan untuk mencari pemisahan terbaik pada atribut kategorikal dengan mengujinya menggunakan skema *one-vs-rest*, yaitu memisahkan data dengan nilai kategori tertentu. Sama seperti fungsi best_split_numerical, akan dipilih atribut yang menghasilkan nilai *entropy* terkecil.

e. find_best_split(X, y)

   Digunakan untuk mengevaluasi seluruh atribut untuk menentukan atribut terbaik yang akan digunakan sebagai *node* pemisah. Fungsi ini akan memeriksa setiap atribut menggunakan fungsi *best split* sesuai tipe atributnya, lalu membandingkan nilai

entropy yang dihasilkan. Atribut dengan *entropy* terendah yang akan dipilih sebagai *node* keputusan.

f. Node(Class)

Merupakan sebuah kelas yang merepresentasikan struktur *node* pada pohon keputusan, di mana setiap *node* dapat menyimpan informasi berupa atribut yang digunakan sebagai pemisah, nilai *threshold*, tipe atribut, dan referensi ke *node* anak kiri dan kanannya. Jika *node* merupakan *leaf*, maka akan dikembalikan nilai prediksi berupa probabilitas kelasnya.

g. build_tree(X, y, depth, max_depth, min_samples, min_samples_leaf)

Digunakan untuk membangun pohon keputusan secara rekursif. Fungsi ini akan berhenti dan membentuk *leaf node* jika seluruh data sudah memiliki kelas, kedalaman maksimumnya tercapai, jumlah data terlalu sedikit, atau pemisahan tidak memenuhi batas minimum jumlah data pada *leaf*. Jika kondisi berhenti tidak terpenuhi, maka fungsi akan mencari atribut terbaik dan membagi data ke *node* kiri dan *node* kanan, kemudian melanjutkan prosesnya secara rekursif.

h. predict_one(row, node)

Digunakan untuk memprediksi satu kelas data dengan menelusuri pohon keputusan dari akar hingga ke *leaf node* berdasarkan nilai atribut pada data. Nilai yang disimpan pada *leaf node* kemudian dikembalikan sebagai hasil prediksinya.

i. predict_proba(X, tree)

Digunakan untuk melakukan prediksi probabilitas pada seluruh data yang ada dengan memanggil *predict_one* untuk setiap baris data dan mengembalikan *array* probabilitas kelas positif. Nilai ini kemudian digunakan untuk mengevaluasi model.

6.2 Algoritma Logistic Regression

Algoritma *Logistic Regression* diimplementasikan *from scratch* menggunakan konsep *gradient descent* untuk optimasi serta menggunakan fungsi sigmoid sebagai fungsi aktivasinya. Algoritma ini dirancang untuk menyelesaikan masalah klasifikasi biner

dengan mempelajari bobot optimal yang meminimalkan *log loss*. Implementasi ini juga mendukung regularisasi L2 dan *class weighting* untuk menangani persebaran data yang tidak seimbang. Berikut merupakan implementasi dari algoritma ini dengan menggunakan bahasa pemrograman Python:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

class CustomLogisticRegression:
    def __init__(self, learning_rate=0.01, n_iterations=1000,
lambda_reg=0.1, use_class_weights=True, verbose=True):
        self.lr = learning_rate
        self.n_iter = n_iterations
        self.lambda_reg = lambda_reg
        self.use_class_weights = use_class_weights
        self.verbose = verbose
        self.weights = None
        self.bias = None
        self.cost_history = []

    def _sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def _compute_cost(self, A, y, m):
        epsilon = 1e-15
        cost_base = - (1/m) * np.sum(y * np.log(A + epsilon) +
(1 - y) * np.log(1 - A + epsilon))
        reg_penalty = (self.lambda_reg / (2 * m)) *
np.sum(np.square(self.weights))
        return cost_base + reg_penalty

    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        if self.use_class_weights:
            unique_classes = np.unique(y)
            class_weights = {}
            for cls in unique_classes:
                count_cls = (y == cls).sum()
                class_weights[cls] = n_samples / (2.0 *
count_cls)
            if self.verbose: print(f"   -> Class Weights
Active: {class_weights}")
```

```
        else:
            if self.verbose: print(f"   -> Class Weights
Disabled (Uniform 1.0)")

        for i in range(self.n_iter):
            linear_model = np.dot(X, self.weights) + self.bias
            A = self._sigmoid(linear_model)
            error = A - y

            if self.use_class_weights:
                sample_weights =
np.array([class_weights[int(y_i)] for y_i in y])
                weighted_error = error * sample_weights
            else:
                weighted_error = error

            dw = (1 / n_samples) * np.dot(X.T, weighted_error)
            db = (1 / n_samples) * np.sum(weighted_error)
            dw += (self.lambda_reg / n_samples) * self.weights
            self.weights -= self.lr * dw
            self.bias -= self.lr * db

            if i % 100 == 0:
                cost = self._compute_cost(A, y, n_samples)
                self.cost_history.append(cost)
                if self.verbose and i % 500 == 0:
                    print(f"      Iter {i}: Cost {cost:.5f}")

        return self

    def predict_proba(self, X):
        X = np.array(X)
        linear_model = np.dot(X, self.weights) + self.bias
        return self._sigmoid(linear_model)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >=
threshold).astype(int)
```

Dapat dilihat bahwa implementasi algoritma *Logistic Regression* ini dilakukan dengan menggunakan *gradient descent* untuk mengoptimalkan bobot model yang meminimalkan *log loss*. Model ini belajar dari data pelatihan dengan iteratif menyesuaikan bobot berdasarkan eror prediksi. Berikut merupakan penjelasan setiap fungsi yang digunakan:

a. CustomLogisticRegression

   Merupakan kelas utama yang mengimplementasikan algoritma Logistic Regression dari scratch. Kelas ini berfungsi menyimpan parameter bobot (weights) dan bias yang

telah dipelajari dari data pelatihan, serta menyediakan metode untuk melakukan prediksi. Parameter utama pada kelas ini meliputi learning rate untuk mengontrol kecepatan pembelajaran, jumlah iterasi training, koefisien regularisasi L2 untuk mencegah overfitting, dan opsi penggunaan class weighting untuk menangani ketidakseimbangan data.

b. sigmoid(z)

   Digunakan untuk mentransformasi nilai logit (kombinasi linear antara input dan bobot) menjadi nilai probabilitas dalam rentang [0, 1]. Pada implementasi ini, dilakukan clipping pada nilai input untuk mencegah instabilitas numerik (overflow atau underflow) saat melakukan perhitungan eksponensial.

c. compute_loss(y_true, y_pred)

   Digunakan untuk menghitung nilai kesalahan (loss) dari prediksi model menggunakan fungsi binary cross-entropy. Jika regularisasi L2 diaktifkan, fungsi ini juga menambahkan penalti terhadap bobot yang terlalu besar untuk mencegah overfitting. Selain itu, prediksi di-clip dengan nilai epsilon kecil untuk menghindari kesalahan perhitungan logaritma pada nilai 0 atau 1.

d. fit(X,y)

   Digunakan untuk melatih model dan memperbarui parameter bobot serta bias menggunakan algoritma gradient descent. Fungsi ini melakukan inisialisasi parameter, perhitungan class weights (jika diaktifkan), dan menjalankan iterasi pembelajaran yang terdiri dari forward pass, perhitungan error, backward pass untuk menghitung gradien, serta pembaruan parameter bobot berdasarkan learning rate.

e. predict_proba(X)

   Digunakan untuk memprediksi probabilitas kelas positif (fraud) pada data uji menggunakan parameter model yang telah dilatih. Fungsi ini melakukan proses forward pass dengan menghitung kombinasi linear input dan bobot, kemudian menerapkannya pada fungsi sigmoid untuk menghasilkan probabilitas.

f. predict(X, threshold=0.5)

   Digunakan untuk menentukan label kelas diskrit (0 atau 1) pada data uji dengan cara mengonversi nilai probabilitas berdasarkan threshold tertentu. Jika probabilitas melebihi threshold, data diklasifikasikan sebagai kelas positif (fraud), dan sebaliknya.

Threshold dapat disesuaikan untuk mengoptimalkan trade-off antara precision dan recall.

6.3 Algoritma K-Nearest Neighbors

Algoritma KNN diimplementasikan *from scratch* dengan pendekatan *lazy learning,* yang mana seluruh data pelatihan disimpan dan proses prediksi dilakukan berdasarkan kedekatan data ui terhadap data latih. Implementasi KNN ini menggunakan metrik jarak *cosine distance* untuk mengukur tingkat kemiripan antar data sehingga prediksi tidak hanya dipengaruhi oleh besar besar nilai fitur, tetapi juga oleh arah vektor fitur. Penentuan kelas dilakukan menggunakan mekanisme *weighted voting,* yang mana kontribusi setiap tetangga diberikan bobot yang berbanding terbalik dengan jaraknya terhadap data uji. Berikut merupakan implementasi algoritma KNN tersebut menggunakan bahasa pemrograman python:

```
import math
from collections import Counter
from sklearn.metrics import classification_report,
roc_auc_score

class KNNClassifier:
    def __init__(self, n_neighbors=11,
distance_metric="cosine", weighted=True):
        self.k = n_neighbors
        self.distance_metric = distance_metric
        self.weighted = weighted
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        self.X_train = np.array(X, dtype=float)
        self.y_train = np.array(y)

    def _get_neighbors(self, x):
        x = np.array(x, dtype=float)
```

```
        if self.distance_metric == "euclidean":
            diff = self.X_train - x
            dists = np.sqrt(np.sum(diff ** 2, axis=1))
        elif self.distance_metric == "manhattan":
            diff = self.X_train - x
            dists = np.sum(np.abs(diff), axis=1)
        elif self.distance_metric == "cosine":
            x_norm = np.linalg.norm(x)
            X_norms = np.linalg.norm(self.X_train, axis=1)
            denom = X_norms * x_norm
            denom = np.where(denom == 0, 1.0, denom)
            sims = (self.X_train @ x) / denom
            sims = np.clip(sims, -1.0, 1.0)
            dists = 1.0 - sims
        else:
            raise ValueError(f"Unknown metric:
{self.distance_metric}")

        dists = np.where(np.isfinite(dists), dists, np.inf)

        k_eff = min(self.k, len(self.X_train))
        idx = np.argpartition(dists, k_eff - 1)[:k_eff]
        return self.y_train[idx], dists[idx]

    def _predict_one(self, x):
        neighbors, neighbor_dists = self._get_neighbors(x)
        if len(neighbors) == 0:
            return 0

        if not self.weighted:
            votes = Counter(neighbors)
            return votes.most_common(1)[0][0]

        eps = 1e-9
        weights = 1.0 / (neighbor_dists + eps)
        weights = np.where(np.isfinite(weights), weights, 0.0)
```

```
        weight_per_label = {}
        for lab, w in zip(neighbors, weights):
            weight_per_label[lab] = weight_per_label.get(lab,
0.0) + w

        max_w = max(weight_per_label.values())
        candidates = [lab for lab, w in
weight_per_label.items() if w == max_w]
        return min(candidates) if candidates else 0

    def _predict_one_proba(self, x):
        neighbors, neighbor_dists = self._get_neighbors(x)
        if len(neighbors) == 0:
            return 0.0

        eps = 1e-9
        weights = 1.0 / (neighbor_dists + eps)
        total_w = np.sum(weights)

        if total_w <= 0 or not math.isfinite(total_w):
            return 0.0

        w_pos = np.sum(weights[neighbors == 1])
        return float(w_pos / total_w)

    def predict(self, X):
        X = np.array(X, dtype=float)
        return np.array([self._predict_one(row) for row in X])

    def predict_proba(self, X):
        X = np.array(X, dtype=float)
        return np.array([self._predict_one_proba(row) for row
in X])

def undersample_majority(X, y, ratio=3, random_state=42):
    rng = np.random.RandomState(random_state)
    idx_pos = np.where(y == 1)[0]
```

```
    idx_neg = np.where(y == 0)[0]
    n_pos = len(idx_pos)
    n_neg_sample = min(len(idx_neg), n_pos * ratio)

    idx_neg_sample = rng.choice(idx_neg, size=n_neg_sample,
replace=False)
    idx_new = np.concatenate([idx_pos, idx_neg_sample])
    rng.shuffle(idx_new)
    return X[idx_new], y[idx_new]

def stratified_split(X, y, test_size=0.2, random_state=42):
    rng = np.random.RandomState(random_state)
    idx = np.arange(len(y))

    idx_pos = idx[y == 1]
    idx_neg = idx[y == 0]
    rng.shuffle(idx_pos); rng.shuffle(idx_neg)

    n_pos_val = int(len(idx_pos) * test_size)
    n_neg_val = int(len(idx_neg) * test_size)

    val_idx = np.concatenate([idx_pos[:n_pos_val],
idx_neg[:n_neg_val]])
    train_idx = np.concatenate([idx_pos[n_pos_val:],
idx_neg[n_neg_val:]])
    rng.shuffle(train_idx); rng.shuffle(val_idx)

    return X[train_idx], X[val_idx], y[train_idx], y[val_idx]


train_raw = train.copy()
test_raw  = test.copy()

X_train_df, y_train, X_test_df, test_ids =
run_universal_pipeline(train_raw, test_raw)

X_all = X_train_df.values
```

```
y_all = y_train.values if hasattr(y_train, "values") else
np.array(y_train)
X_test_all = X_test_df.values


X_bal, y_bal = undersample_majority(X_all, y_all, ratio=3,
random_state=42)
X_tr, X_val, y_tr, y_val = stratified_split(X_bal, y_bal,
test_size=0.2, random_state=42)

k_val = 11 #coba2 k di bagian ini
knn_model = KNNClassifier(n_neighbors=k_val,
distance_metric="cosine", weighted=True)
knn_model.fit(X_tr, y_tr)

y_val_pred  = knn_model.predict(X_val)
y_val_proba = knn_model.predict_proba(X_val)

print(classification_report(y_val, y_val_pred, digits=4))
print("ROC AUC:", roc_auc_score(y_val, y_val_proba))

knn_model.fit(X_bal, y_bal)

test_scores = knn_model.predict_proba(X_test_all)
test_preds  = (test_scores >= 0.5).astype(int)

submission = pd.DataFrame({
    "ID": test_ids.astype(int),
    "is_fraud": test_preds.astype(int)
})

print("Unique is_fraud values:",
submission["is_fraud"].unique())
print(submission.head())

submission.to_csv("submission_knn_from_scratch.csv",
index=False)
```

```
print("Submission saved as submission_knn_from_scratch.csv")
from google.colab import files
files.download("submission_knn_from_scratch.csv")
```

Dapat dilihat bahwa implementasi algoritma K-Nearest Neighbors (KNN) ini dilakukan dengan menyimpan seluruh data pelatihan dan melakukan proses prediksi berdasarkan kedekatan data uji terhadap data latih menggunakan metrik jarak tertentu. Untuk setiap data uji, jarak terhadap seluruh data pelatihan dihitung, kemudian dipilih sebanyak k tetangga terdekat. Penentuan kelas dilakukan menggunakan mekanisme *weighted voting*. Berikut merupakan penjelasan dari setiap fungsi yang digunakan:

a. KNNClassifier

   Merupakan kelas utama yang mengimplementasikan algoritma KNN dari *scratch.* Kelas ini menyimpan data pelatihan dan menyediakan metode untuk melakukan prediksi kelas maupun probabilitas berdasarkan data uji dengan data latih. Parameter utama pada kelas ini adalah jumlah tetangga terdekat k, metrik jarak yang digunakan, serta opsi penggunaan *weighted voting.*

b. fit(X,y)

   Digunakan untuk menyimpan data pelatihan dan label target ke dalam model. Pada algoritma KNN, fungsi ini tidak melakukan proses pembelajaran parameter, melainkan hanya menyimpan data latih yang nantinya akan digunakan pada tahap prediksi (*lazy learning).*

c. _get_neighbors(x)

   Digunakan untuk mencari tetangga terdekat dari sebuah data uji x. Fungsi ini menghitung jarak antara x dan seluruh data pelatihan menggunakan metrik jarak yang dipilih, yaitu *Euclidean Distance, Manhattan Distance,* atau *Cosine Distance.* Pada implementasi ini, *Cosine Distance* digunakan dengan menghitung nilai kemiripan *cosinus* antar vektor, kemudian mengubahnya menjadi jarak. Setelah jarak dihitung,

fungsi ini memilih sebanyak k tetangga dengan jarak terkecil dan mengembalikan label serta jarak dari tetangga tersebut.

d. _predict_one(X)

Digunakan untuk memprediksi kelas dari satu data uji dengan mekanisme *weighted voting*. Setiap tetangga diberikan bobot yang berbanding terbalik dengan jaraknya terhadap data uji, sehingga tetangga yang lebih dekat memiliki pengaruh yang lebih besar. Bobot untuk masing-masing kelas dijumlahkan, dan kelas dengan total bobot terbesar dipilih sebagai hasil prediksi akhir.

e. _predict_proba(X)

Digunakan untuk menghasilkan skor probabilitas *fraud* untuk seluruh data uji. Fungsi ini memanggil _predict_one_proba untuk setiap baris data dan mengembalikan array probabilitas yang kemudian dapat digunakan untuk evaluasi model.

f. undersample_majority(X, y, ratio)

Digunakan untuk menangani permasalahan ketidakseimbangan kelas pada dataset. Fungsi ini mempertahankan seluruh data *fraud* dan melakukan pengambilan sampel secara acak pada kelas *non-fraud* dengan rasio *fraud:non-fraud* sebesar 1:3. Tujuan dari proses ini adalah untuk mengurangi bias model terhadap kelas *fraud* dan meningkatkan kemampuan model dalam mendeteksi transaksi *fraud.*

g. stratified_split(X, y, test_size)

Digunakan untuk membagi data menjadi data pelatihan dan data validasi dengan mempertahankan proporsi kelas pada masing-masing subset. Proses ini memastikan distribusi kelas pada data validasi tetap representatif sehingga evaluasi model dapat dilakukan secara lebih objektif.

Setelah data diproses, model KNN dilatih dengan menggunakan data pelatihan dan dievaluasi menggunakan data validasi. Evaluasi dilakukan dengan menggunakan *classification report* untuk melihat nilai *precision, recall,* dan F1-*score,* serta ROC AUC yang dihitung berdasarkan skor probabilitas. Setelah evaluasi selesai, model dilatih ulang menggunakan seluruh data pelatihan yang telah diseimbangkan, yang akan digunakan

untuk memprediksi data uji. Hasil prediksi probabilitas selanjutnya dikonversi menjadi label kelas biner dan disimpan ke dalam *file submission.*

BAB VII

DATA *SPLITTING*, *PREPROCESSING PIPELINE*, *SUBMISSION*, dan BONUS

7.1 Data *Splitting*

Data *splitting* atau pembagian dataset dilakukan untuk memisahkan data pelatihan menjadi *training* set dan *validation* set. Hal ini dilakukan untuk mencegah terjadinya data *leakage*. Pada tugas besar ini, data pelatihan dibagi dengan rasio 80% untuk *training* set dan 20% untuk *validation* set secara acak menggunakan *random state* agar hasil pembagiannya bersifat *reproducible*. *Training* set akan digunakan untuk melatih model, sedangkan *validation* set digunakan untuk mengevaluasi model. Berikut merupakan implementasi dari data *splitting*:

```
original_train_set = train.copy()
# Rasio validation size, yaitu 20%
val_size = 0.2
random_state = 31

# Split jadi train_set dan val_set
train_set, val_set = train_test_split(
    train,
    test_size=val_size,
    random_state=random_state,
    shuffle=True
)

print("Train size:", len(train_set))
print("Validation size:", len(val_set))
```

7.2 Preprocessing Pipeline

Preprocessing pipeline merupakan tahapan yang dilakukan untuk menggabungkan seluruh proses pembersihan dan *preprocessing* data yang telah didefinisikan sebelumnya agar alur pemrosesannya menjadi lebih terstruktur. Proses ini mencakup penghapusan

data duplikat, penanganan nilai hilang, *feature engineering*, penanganan *outliers*, *encoding* fitur kategorikal, dan scaling fitur numerik. Penyusunan ini membuat proses *preprocessing* dapat dilakukan secara modular dan fleksibel. Berikut merupakan implementasi dari *preprocessing pipeline* yang dibuat:

```
def run_universal_pipeline(df_train_raw, df_test_raw):
    CFG_REMOVE_DUPLICATES = True
    CFG_IMPUTE_MISSING    = True
    CFG_FEATURE_ENG       = True
    CFG_OUTLIER_HANDLING  = True
    CFG_ENCODING          = True
    CFG_SCALING           = True
    CFG_SMOTE             = False
    CFG_PCA               = False

    # INITIALIZATION
    drop_list, bin_list, nom_list, num_list =
classify_features(df_train_raw)
    test_ids = df_test_raw['ID'] if 'ID' in df_test_raw.columns
else df_test_raw.index

    X_train = df_train_raw.drop(columns=drop_list,
errors='ignore')
    y_train = X_train.pop('is_fraud')
    X_test = df_test_raw.drop(columns=drop_list, errors='ignore')
    if 'is_fraud' in X_test.columns: X_test =
X_test.drop(columns=['is_fraud'])

    # DATA CLEANING
    if CFG_REMOVE_DUPLICATES:
        X_train = remove_duplicates(X_train)
        y_train = y_train.loc[X_train.index]

    if CFG_IMPUTE_MISSING:
        X_train, X_test = impute_missing_values(X_train, X_test,
num_list, nom_list, bin_list)

    if CFG_FEATURE_ENG:
```

```
        X_train, X_test, num_list = generate_new_features(X_train,
X_test, num_list)

    if CFG_OUTLIER_HANDLING:
        X_train, X_test = handle_outliers_and_skew(X_train,
X_test, num_list)

    # DATA PREPROCESSING
    if CFG_ENCODING:
        X_train, X_test = perform_encoding(X_train, X_test)

    if CFG_SCALING:
        X_train, X_test = perform_scaling(X_train, X_test)

    if CFG_SMOTE: # Skip jika False
        X_train, y_train = perform_resampling(X_train, y_train)

    if CFG_PCA: # Skip jika False
        X_train, X_test = perform_dim_reduction(X_train, X_test,
y_train)

    return X_train, y_train, X_test, test_ids
```

7.3 Submission

Tahapan ini dilakukan untuk menyimpan hasil prediksi ke dalam berkas CSV yang berisi kolom ID dan *is_fraud* yang sesuai dengan format yang telah ditentukan untuk proses submission. Berikut adalah implementasinya:

```
def coba_satu_satu(mode, algo, ratio=1.5):
    global MODE, ALGO_TO_RUN
    MODE = mode
    ALGO_TO_RUN = algo

    train =
pd.read_csv("https://drive.google.com/uc?id=1BFj1nqZ0pJXraLv4W2N_T
a8tWtNT_z3o")
```

```
    test  =
pd.read_csv("https://drive.google.com/uc?id=1yKJ_dPO7mjEDTdiskt2f4
5sN4aHyFsKV")

    X_train_clean, y_train_clean, X_test_clean, ids_clean =
run_universal_pipeline(train, test)
    preds_dict = run_evaluation_cycle(X_train_clean,
y_train_clean, X_test_clean, ids_clean, [algo])

    final_prob = preds_dict[algo]
    filename = f"submission_{algo}_{mode}.csv"

    pd.DataFrame({
        "ID": ids_clean,
        "is_fraud": final_prob
    }).to_csv(filename, index=False)

    print(f"[DONE] {algo} | MODE={mode} |
MeanProb={final_prob.mean():.4f}")
    print(f"\n File '{filename}' siap membantai leaderboard
private :D")

    return None
```

Bagian tersebut mengeksekusi pipeline end-to-end untuk satu kombinasi mode dan algoritma dari loading data hingga menghasilkan file prediksi, contoh: menjalankan coba_satu_satu('SCRATCH', 'KNN') akan melatih KNN scratch, mengevaluasi, lalu menghasilkan probabilitas prediksi test untuk submission.

7.4 Bonus DTL

Proses visualisasi dimulai dari *root* hingga *leaf* dengan mengikuti percabangan kiri dan kanan pada setiap *node*. Setiap *node* internal menampilkan aturan *split* berdasarkan fitur dan *threshold*, sedangkan *leaf node* menampilkan probabilitas *fraud* sebagai hasil prediksi. Parameter *max_depth* digunakan untuk membatasi kedalaman *tree* yang ditampilkan agar visualisasi tetap ringkas dan mudah dibaca, parameter ini dapat

diubah-ubah nilainya sesuai keinginan. *Node* yang melewati batas kedalaman akan ditandai sebagai *leaf* terpotong (*truncated*).

```
# Hapus tanda pagar (#) pada line 1 dan 2 untuk menginstall
graphviz
# !apt-get install graphviz -y
# !pip install graphviz

from graphviz import Digraph

def tree_png(node, max_depth=3, depth=0, graph=None, node_id=0):
    if graph is None:
        graph = Digraph(format="png")
        graph.attr(rankdir="TB", size="8,10")

    idx = str(node_id)

    if node.value is not None:
        label = f"Leaf\nP(fraud)={node.value:.3f}"
        graph.node(idx, label=label, shape="box", style="filled",
fillcolor="skyblue")
        return graph, node_id + 1

    if depth >= max_depth:
        label = "Leaf\n(truncated)"
        graph.node(idx, label=label, shape="box", style="filled",
fillcolor="lightpink")
        return graph, node_id + 1

    if node.feature_type == "numeric":
        label = f"{node.feature}\n<= {node.threshold:.3f}"
    else:
        label = f"{node.feature}\n== {node.threshold}"

    graph.node(idx, label=label)

    # Cabang kanan
    graph, next_idx = tree_png(node.left, max_depth, depth + 1,
graph, node_id + 1)
```

```
    graph.edge(idx, str(node_id + 1), label="Yes")

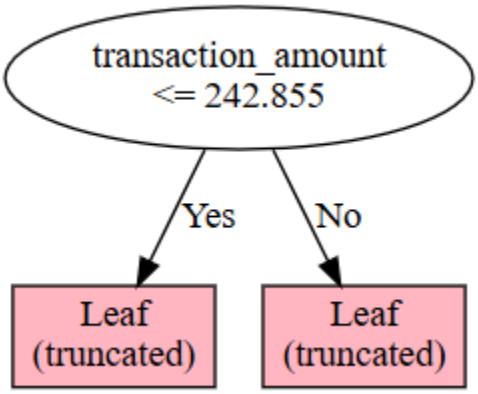
    # Cabang kiri
    graph, next_idx = tree_png(node.right, max_depth, depth + 1,
graph, next_idx)
    graph.edge(idx, str(next_idx - 1), label="No")

    return graph, next_idx

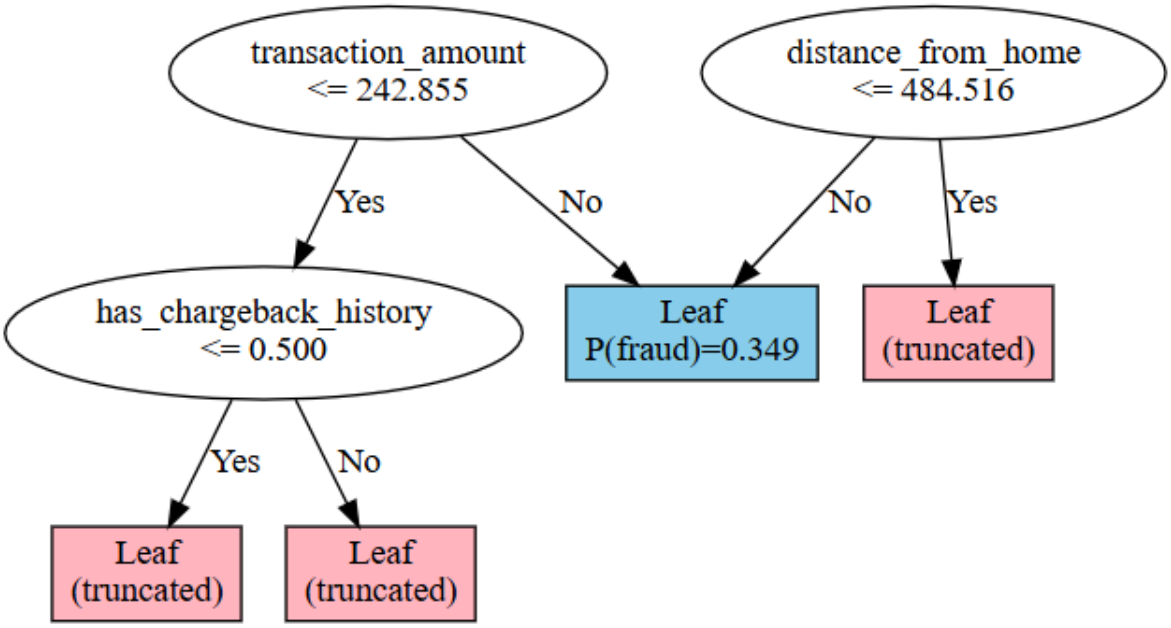
graph, _ = tree_png(tree, max_depth=4)
graph
```

Berikut adalah hasilnya:

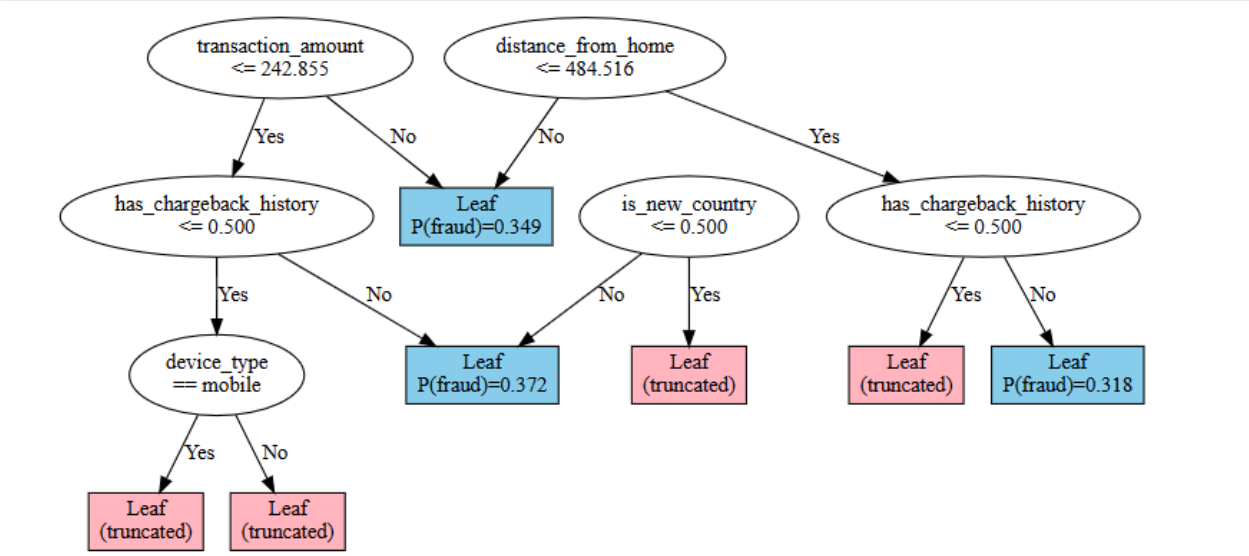
- Kedalaman 1



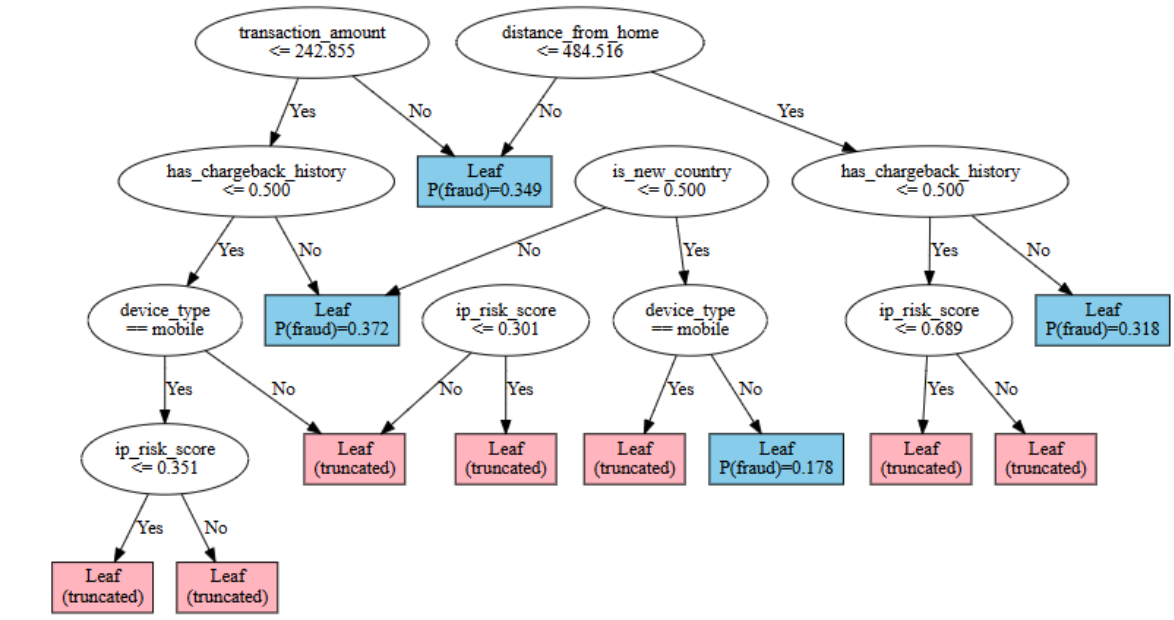
- Kedalaman 2



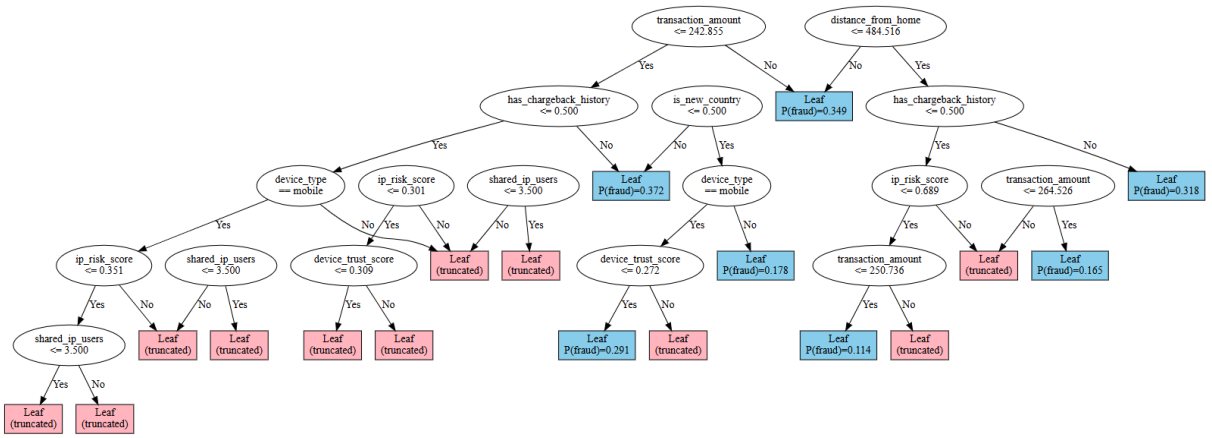
- Kedalaman 3



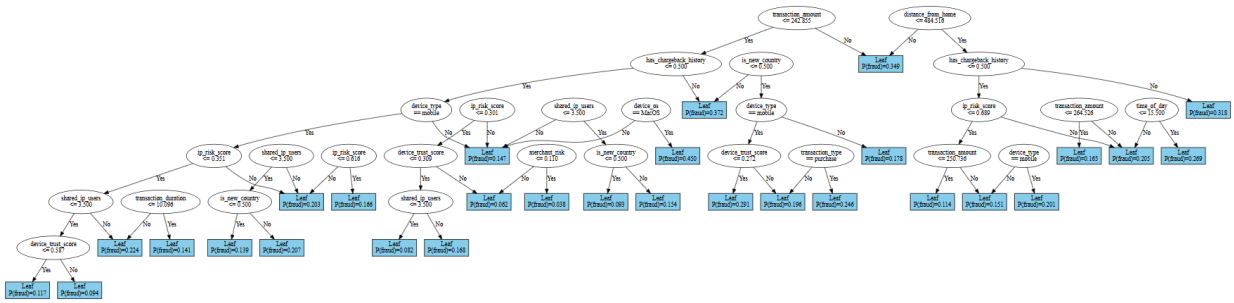
- Kedalaman 4



- Kedalaman 5



- Kedalaman 6 (max_depth)



BAB VIII
PERBANDINGAN HASIL PREDIKSI

7.1 Perbandingan Hasil Prediksi Algoritma Decision Tree Learning

From Scratch	precision recall f1-score support 0 0.8593 0.9907 0.9204 17174 1 0.2050 0.0145 0.0271 2826 accuracy 0.8528 20000 macro avg 0.5322 0.5026 0.4737 20000 weighted avg 0.7669 0.8528 0.7942 20000 ---------- AUC : 0.5748 Runtime : 250.79 s ----------
With Library (Sckit)	precision recall f1-score support 0 0.8600 0.9871 0.9192 17174 1 0.2318 0.0237 0.0430 2826 accuracy 0.8509 20000 macro avg 0.5459 0.5054 0.4811 20000 weighted avg 0.7713 0.8509 0.7954 20000 ---------- AUC : 0.5665 Runtime : 2.60 s ----------

Berdasarkan hasil perbandingan tersebut, implementasi algoritma Decision Tree Learning baik **from scratch** maupun menggunakan **library (Scikit-learn)** menunjukkan performa yang relatif serupa. Kedua model mencapai tingkat akurasi sekitar 85%, yang didominasi oleh kemampuan dalam mengklasifikasikan kelas *non-fraud* dengan baik, dilihat dari nilai precision dan recall yang tinggi pada kelas tersebut. Namun, performa pada kelas *fraud* masih rendah, khususnya pada nilai recall dan F1-score, yang menandakan bahwa sebagian besar kasus fraud masih sulit terdeteksi.

Meskipun demikian, implementasi berbasis library menunjukkan sedikit peningkatan pada metrik kelas fraud serta waktu eksekusi yang jauh lebih cepat dibandingkan implementasi from scratch. Di sisi lain, nilai ROC AUC pada model from scratch sedikit lebih tinggi, menunjukkan kemampuan pemisahan kelas yang sebanding secara keseluruhan. Hasil ini mengindikasikan bahwa implementasi from scratch telah berhasil merepresentasikan konsep dasar Decision Tree Learning dengan baik, meskipun library tetap unggul dalam hal efisiensi dan stabilitas komputasi.

7.2 Perbandingan Hasil Prediksi Algoritma Logistic Regression

From Scratch	precision recall f1-score support 0 0.8841 0.5945 0.7110 17174 1 0.1761 0.5265 0.2639 2826 accuracy 0.5849 20000 macro avg 0.5301 0.5605 0.4874 20000 weighted avg 0.7841 0.5849 0.6478 20000 ---------- AUC : 0.5911 Runtime : 156.63 s ----------
With Library (Sklearn)	precision recall f1-score support 0 0.8587 0.9999 0.9239 17174 1 0.3333 0.0004 0.0007 2826 accuracy 0.8587 20000 macro avg 0.5960 0.5001 0.4623 20000 weighted avg 0.7845 0.8587 0.7935 20000 ---------- AUC : 0.5927 Runtime : 0.47 s ----------

Berdasarkan hasil tersebut, algoritma Logistic Regression yang diimplementasikan from scratch dan menggunakan library (Scikit-learn) menunjukkan karakteristik performa yang berbeda. Implementasi from scratch memiliki recall kelas fraud yang jauh lebih tinggi

dibandingkan versi library, yang menunjukkan bahwa model ini lebih sensitif dalam mendeteksi transaksi fraud. Namun, peningkatan recall tersebut diikuti dengan precision yang rendah, sehingga banyak prediksi fraud yang tidak tepat. Hal ini berdampak pada nilai akurasi keseluruhan yang lebih rendah dibandingkan implementasi berbasis library.

Sebaliknya, implementasi Logistic Regression menggunakan library menghasilkan akurasi yang lebih tinggi, tetapi hampir seluruh prediksi difokuskan pada kelas mayoritas (non-fraud), yang terlihat dari nilai recall kelas fraud yang sangat rendah. Meskipun nilai ROC AUC kedua pendekatan relatif mirip, perbedaan yang signifikan terlihat pada waktu eksekusi, di mana implementasi library jauh lebih efisien. Secara keseluruhan, hasil ini menunjukkan adanya trade-off antara kemampuan mendeteksi kelas minoritas dan stabilitas performa model, serta menegaskan bahwa implementasi from scratch berhasil merepresentasikan konsep dasar algoritma meskipun masih kalah dalam efisiensi komputasi.

7.3 Perbandingan Hasil Prediksi Algoritma K-Nearest Neighbors

From Scratch	🔹 Processing: KNN... precision recall f1-score support 0 0.8593 0.9966 0.9228 17174 1 0.2805 0.0081 0.0158 2826 accuracy 0.8569 20000 macro avg 0.5699 0.5024 0.4693 20000 weighted avg 0.7775 0.8569 0.7947 20000 ---------- AUC : 0.5323 Runtime : 480.70 s ----------

With Library (Sklearn)	precision recall f1-score support 0 0.8587 0.9999 0.9239 17174 1 0.0000 0.0000 0.0000 2826 accuracy 0.8586 20000 macro avg 0.4293 0.4999 0.4620 20000 weighted avg 0.7374 0.8586 0.7934 20000 --- AUC : 0.5316 Runtime : 118.71 s ---

Berdasarkan tabel tersebut, algoritma K-Nearest Neighbors (KNN) yang diimplementasikan from scratch dan menggunakan library (Scikit-learn) menunjukkan pola performa yang serupa, khususnya dalam menangani kelas mayoritas (non-fraud). Kedua pendekatan memiliki nilai akurasi yang relatif tinggi (sekitar 85%), yang terutama dipengaruhi oleh nilai recall kelas non-fraud yang sangat tinggi. Namun, performa pada kelas minoritas (fraud) masih sangat rendah, terutama pada nilai recall dan F1-score, yang menunjukkan bahwa sebagian besar transaksi fraud gagal terdeteksi oleh model KNN. Perbedaan utama terlihat pada aspek efisiensi komputasi, yang mana implementasi from scratch memiliki waktu eksekusi yang jauh lebih lama dibandingkan versi library. Hal ini disebabkan oleh sifat KNN sebagai algoritma berbasis jarak yang memerlukan perhitungan jarak ke seluruh data pelatihan pada saat prediksi, serta kurangnya optimasi internal pada implementasi manual. Meskipun nilai ROC AUC kedua pendekatan relatif mirip dan cenderung rendah, hasil ini menunjukkan bahwa implementasi KNN from scratch telah berhasil merepresentasikan mekanisme dasar algoritma sesuai teori, meskipun secara praktis masih kurang efektif untuk mendeteksi kelas fraud pada dataset yang tidak seimbang.

BAB IX
PENUTUP

8.1 Kesimpulan

Berdasarkan seluruh tahapan analisis dan pemodelan yang telah dilakukan dalam notebook ini, dapat disimpulkan bahwa proses deteksi penipuan transaksi telah dilaksanakan secara sistematis dan menyeluruh, mulai dari tahap pra-pemrosesan data hingga tahap evaluasi dan prediksi akhir.

1. **Pra-pemrosesan Data (Preprocessing)**

   Tahap pra-pemrosesan diawali dengan pembersihan data, yaitu penanganan nilai hilang, data duplikat, serta outlier yang berpotensi mengganggu proses pembelajaran model. Selanjutnya, dilakukan proses *feature engineering* dengan membangun fitur-fitur baru yang bertujuan untuk meningkatkan kemampuan model dalam membedakan transaksi penipuan dan non-penipuan, salah satunya adalah fitur *risk_log_amount* yang memanfaatkan transformasi logaritmik untuk menstabilkan varians pada nilai *transaction_amount* yang dikombinasikan dengan *ip_risk_score*.

   Selain itu, fitur numerik dilakukan penskalaan menggunakan metode seperti *StandardScaler* agar berada pada rentang nilai yang sebanding, sedangkan fitur kategorikal dikonversi ke dalam bentuk numerik melalui teknik *Label Encoding* atau *One-Hot Encoding*. Untuk mengatasi permasalahan ketidakseimbangan kelas antara transaksi penipuan dan non-penipuan, diterapkan atau dipertimbangkan teknik penanganan ketidakseimbangan data, seperti *undersampling*.

2. **Pemodelan (Modeling)**

   Pada tahap pemodelan, tiga algoritma utama digunakan dan dibandingkan, yaitu *Decision Tree Learning (DTL)*, *Logistic Regression*, dan *K-Nearest Neighbors (KNN)*. Masing-masing algoritma diimplementasikan dalam dua pendekatan, yaitu implementasi dari awal (*scratch*) dan implementasi menggunakan pustaka *scikit-learn*. Seluruh model dilatih menggunakan data latih yang telah melalui proses pra-pemrosesan.

3. **Evaluasi Model**

   Evaluasi kinerja model dilakukan menggunakan beberapa metrik, antara lain *Accuracy*, *Precision*, *Recall*, *F1-Score*, dan *ROC-AUC*. Hasil evaluasi menunjukkan perbedaan kinerja antar model dalam mendeteksi transaksi penipuan. Meskipun nilai numerik spesifik dari setiap metrik bergantung pada hasil eksekusi kode terakhir, proses evaluasi ini bertujuan untuk menentukan model dengan performa terbaik. Secara umum, model yang diimplementasikan menggunakan pustaka cenderung memiliki waktu eksekusi yang lebih efisien dan performa yang lebih stabil dibandingkan dengan implementasi *scratch*, meskipun implementasi *scratch* memberikan pemahaman yang lebih mendalam terhadap mekanisme kerja algoritma.

4. **Submission**

   Model yang dipilih sebagai model terbaik kemudian dilatih ulang menggunakan seluruh data latih dan digunakan untuk menghasilkan prediksi pada data uji. Hasil prediksi tersebut disusun dalam format yang sesuai untuk proses *submission*, yaitu mencakup kolom *id* dan *is_fraud*.

8.2 Kontribusi Setiap Anggota Kelompok

Tabel 5 Kontribusi Setiap Anggota Kelompok

NIM	Nama Anggota	Pembagian Tugas
18222070	Favian Izza Diasputra	Implementasi: ● Mengerjakan feature encoding ● Mengerjakan feature scaling ● Mengerjakan data normalization ● Mengerjakan Exploratory Data Analysis ● Mengerjakan compile preprocessing pipeline ● Mengerjakan algoritma DTL, Logistic Regression, dan KNN menggunakan scikit-learn ● Mengerjakan submissions Laporan:

		• Mengerjakan data information • Mengerjakan Exploratory Data Analysis • Mengerjakan cleaning and preprocessing • Mengerjakan perbandingan hasil prediksi
18223008	Henrycus Hugatama Risaldy	Implementasi: • Mengerjakan feature engineering • Mengerjakan dimensionality reduction • Mengerjakan algoritma Logistic Regression from scratch Laporan: • Mengerjakan dasar teori algoritma Logistic Regression • Mengerjakan implementasi algoritma Logistic Regression
18223046	Farella Kamala Budianto	Implementasi: • Mengerjakan split training, validation, and testing set • Mengerjakan handling missing data • Mengerjakan Exploratory Data Analysis • Mengerjakan algoritma K-Nearest Neighbors from scratch Laporan: • Mengerjakan latar belakang • Mengerjakan dasar teori algoritma K-Nearest Neighbors • Mengerjakan cleaning and preprocessing • Mengerjakan implementasi algoritma K-Nearest Neighbors
18223098	Kenlyn Tesalonika Winata	Implementasi: • Mengerjakan dealing with outliers • Mengerjakan remove duplicates

		● Mengerjakan handling imbalance dataset ● Mengerjakan algoritma DTL from scratch Laporan: ● Mengerjakan dasar teori algoritma DTL ● Mengerjakan cleaning and preprocessing ● Mengerjakan implementasi algoritma DTL ● Mengerjakan data splitting, preprocessing pipeline, dan submission

REFERENSI

AmirMasoud. (2022, June 19). *Understanding micro, macro, and weighted averages for scikit-learn metrics in multi-class classification (with example)*. https://iamirmasoud.com/2022/06/19/understanding-micro-macro-and-weighted-averages-for-scikit-learn-metrics-in-multi-class-classification-with-example/

Arize AI. (2023, Maret 16). *Deep dive on KNN: Understanding and implementing the K-Nearest Neighbors algorithm* [Blog post]. Arize AI. https://arize.com/blog-course/knn-algorithm-k-nearest-neighbor/

DataCamp. (2023, Juni 14). *K-Nearest Neighbors (KNN) classification with R tutorial*. DataCamp. https://www.datacamp.com/tutorial/k-nearest-neighbors-knn-classification-with-r-tutorial

Duda, R. O., Hart, P. E., & Stork, D. G. (2001). *Pattern classification* (2nd ed.). John Wiley & Sons. (Chapter 5)

Elkan, C. (2014). *Maximum likelihood, logistic regression, and stochastic gradient training*. https://cseweb.ucsd.edu/~elkan/250B/logreg.pdf

GeeksforGeeks. (n.d.). *Cosine similarity*. GeeksforGeeks. https://www.geeksforgeeks.org/dbms/cosine-similarity/

GeeksforGeeks. (2021, November 17). *Euclidean distance – Understanding the math with examples*. GeeksforGeeks. https://www.geeksforgeeks.org/euclidean-distance/

GeeksforGeeks. (2025, Agustus 23). *K-Nearest Neighbor (KNN) Algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/

GeeksforGeeks. (2023, July 31). *Manhattan Distance in Machine Learning*. GeeksforGeeks. https://www.geeksforgeeks.org/data-science/manhattan-distance/

Géron, A. (2023). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). GitHub repository. https://github.com/ageron/handson-ml3

Khodra, M. L. (n.d.). *Logistic regression: Stochastic gradient ascent* [Slide presentasi]. Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

k-nearest neighbors algorithm. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm

Maulidevi, N. U. (n.d.). *Decision tree learning (DTL)* [Slide presentasi]. Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Maulidevi, N. U. (n.d.). *k-Nearest Neighbor* [Slide presentasi]. Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Maulidevi, N. U. (n.d.). *Prediction Measurement* [Slide presentasi]. Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Murphy, K. P. (2022). *Probabilistic machine learning: An introduction*. MIT Press. https://probml.github.io/pml-book/book1.html

Russell, S., & Norvig, P. (2010). *Artificial intelligence: A modern approach* (3rd ed., Chapter 18.6.4). Prentice Hall.

Studocu. (n.d.). *PA unit 3: Decision trees and logistic regression in predictive analytics*. https://www.studocu.com/in/document/jawaharlal-nehru-technological-university-hyderabad/predictive-analytics/pa-unit-3-decision-trees-and-logistic-regression-in-predictive-analytics/148008643

Supe, K. (2023, June 7). Understanding cosine similarity in Python with scikit-learn. Memgraph. https://memgraph.com/blog/cosine-similarity-python-scikit-learn

Walpole, R. E., Myers, R. H., Myers, S. L., & Ye, K. (2012). *Probability and statistics for engineering and the sciences* (9th ed., pp. 430–435, Chs. 11, 12.12, 9.14). Pearson Education.

Yaqoob, N. (2023, Oktober 17). *K-nearest neighbors algorithm for credit card fraud detection*. PyFi. https://pyfi.com/blogs/articles/k-nearest-neighbors-algorithm-for-credit-card-fraud-detection